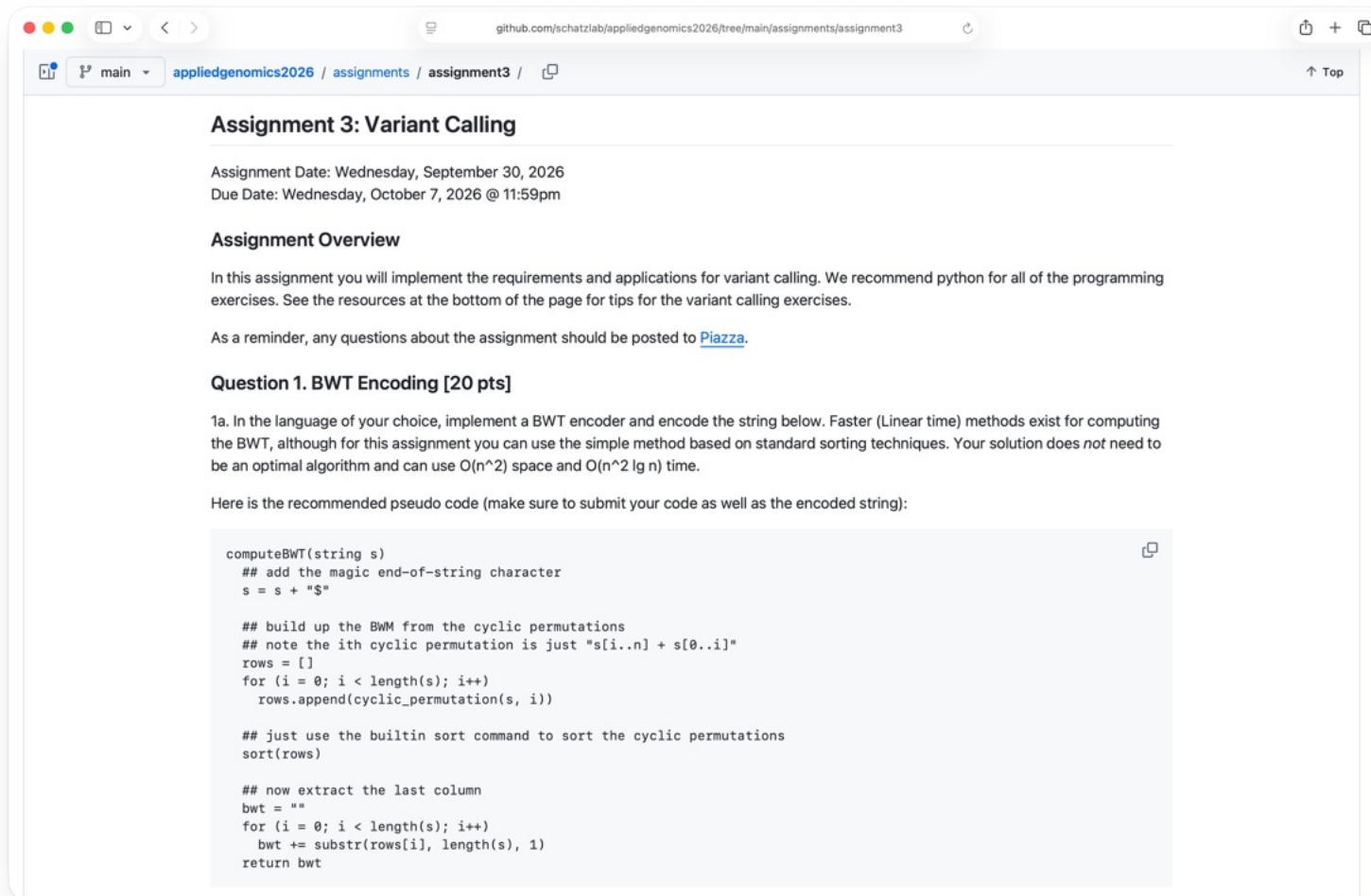


ImageNet Classification with Deep Convolutional Neural Networks

Michael Schatz
Oct 5, 2026

JHU EN.601.449/EN.601.649: Applied Comparative Genomics

Assignment 3 Due Wednesday Oct 7 by 11:59pm



The screenshot shows a web browser displaying a GitHub repository page. The browser's address bar shows the URL: `github.com/schatzlab/appliedgenomics2026/tree/main/assignments/assignment3`. The repository name is `appliedgenomics2026`, and the specific file or directory is `assignments/assignment3`. The page title is **Assignment 3: Variant Calling**. Below the title, the assignment date is listed as Wednesday, September 30, 2026, and the due date is Wednesday, October 7, 2026 @ 11:59pm. The section **Assignment Overview** explains that the assignment involves implementing requirements and applications for variant calling, recommending Python. It also mentions that questions should be posted to Piazza. The section **Question 1. BWT Encoding [20 pts]** asks the user to implement a BWT encoder. It provides a detailed description of the task and a recommended pseudo-code snippet for computing the BWT.

Assignment 3: Variant Calling

Assignment Date: Wednesday, September 30, 2026
Due Date: Wednesday, October 7, 2026 @ 11:59pm

Assignment Overview

In this assignment you will implement the requirements and applications for variant calling. We recommend python for all of the programming exercises. See the resources at the bottom of the page for tips for the variant calling exercises.

As a reminder, any questions about the assignment should be posted to [Piazza](#).

Question 1. BWT Encoding [20 pts]

1a. In the language of your choice, implement a BWT encoder and encode the string below. Faster (Linear time) methods exist for computing the BWT, although for this assignment you can use the simple method based on standard sorting techniques. Your solution does *not* need to be an optimal algorithm and can use $O(n^2)$ space and $O(n^2 \lg n)$ time.

Here is the recommended pseudo code (make sure to submit your code as well as the encoded string):

```
computeBWT(string s)
  ## add the magic end-of-string character
  s = s + "$"

  ## build up the BWM from the cyclic permutations
  ## note the ith cyclic permutation is just "s[i..n] + s[0..i]"
  rows = []
  for (i = 0; i < length(s); i++)
    rows.append(cyclic_permutation(s, i))

  ## just use the builtin sort command to sort the cyclic permutations
  sort(rows)

  ## now extract the last column
  bwt = ""
  for (i = 0; i < length(s); i++)
    bwt += substr(rows[i], length(s), 1)
  return bwt
```

Exam 1 Next Wednesday

12	10/12/26	M	Review 1	Project Proposal	
13	10/14/26	W	Exam 1 (In class)		

AlexNext pt 2

“AlexNet”

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called “dropout” that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.



Alex Krizhevsky
U. Toronto/Google



Ilya Sutskever
U. Toronto/OpenAI/
Safe Superintelligence Inc



Geoffrey Hinton
U. Toronto/Vector Institute

What is this?



Fish



Axe



Bagel

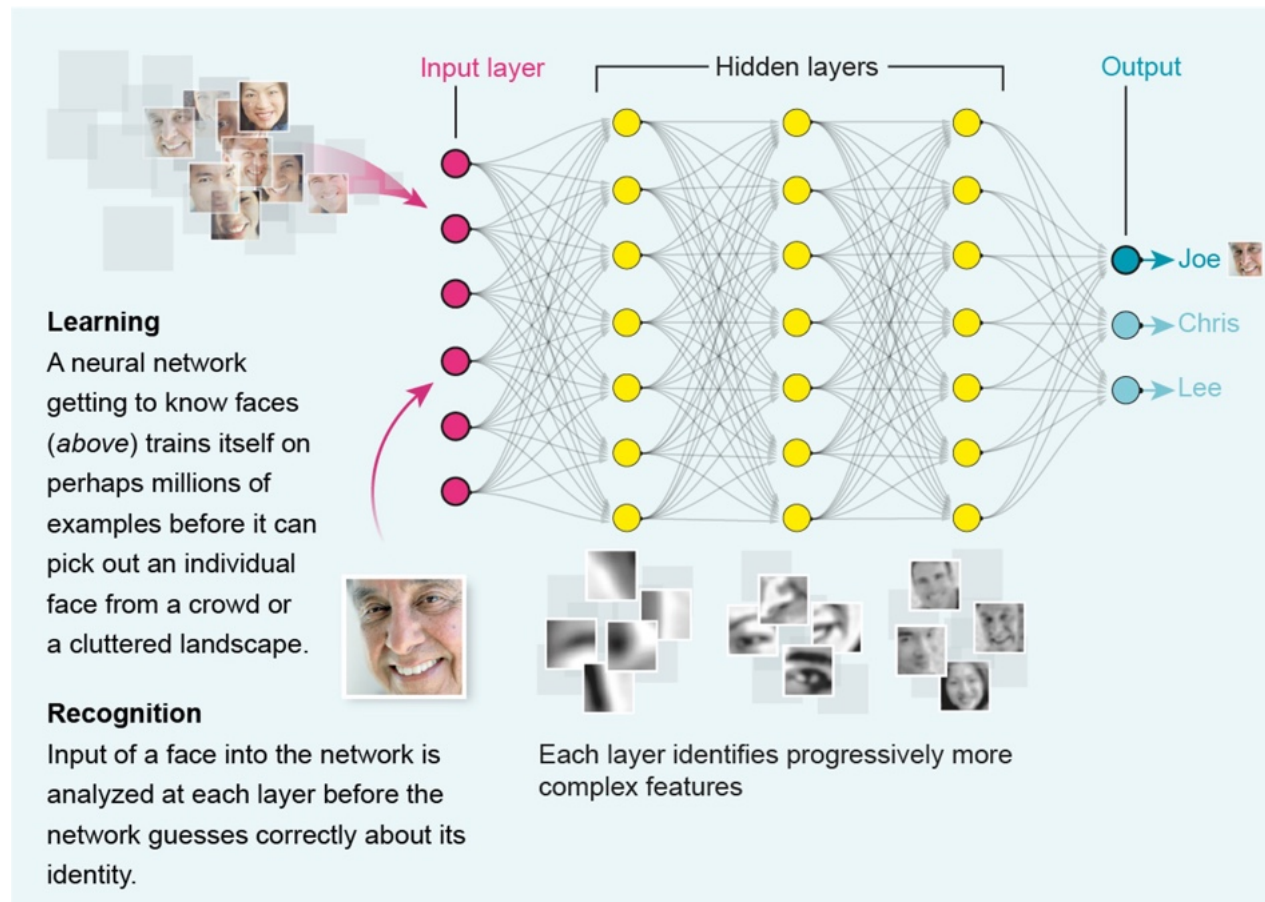


Dog



Van

Artificial Neural Networks



<https://www.codingdojo.com/blog/artificial-neural-network-brain-behind-todays-smart-technology>

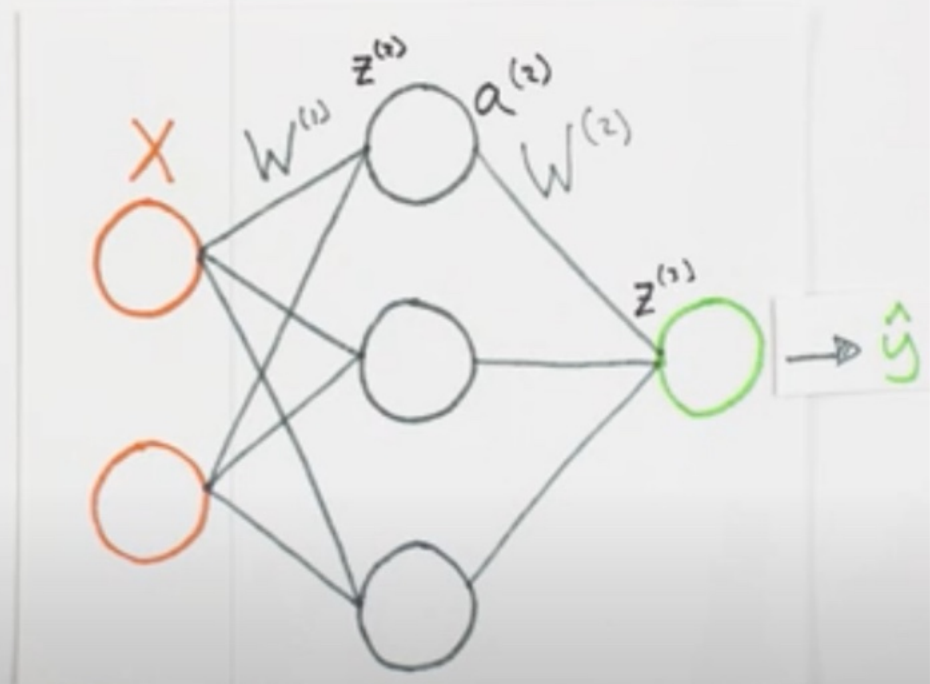
Training with Backpropagation: Random initialization

```
class Neural_Network(object):
    def __init__(self):
        #Define Hyperparameters
        self.inputLayerSize = 2
        self.outputLayerSize = 1
        self.hiddenLayerSize = 3

        #Weights (Parameters)
        self.W1 = np.random.randn(self.inputLayerSize, \
                                    self.hiddenLayerSize)
        self.W2 = np.random.randn(self.hiddenLayerSize, \
                                    self.outputLayerSize)

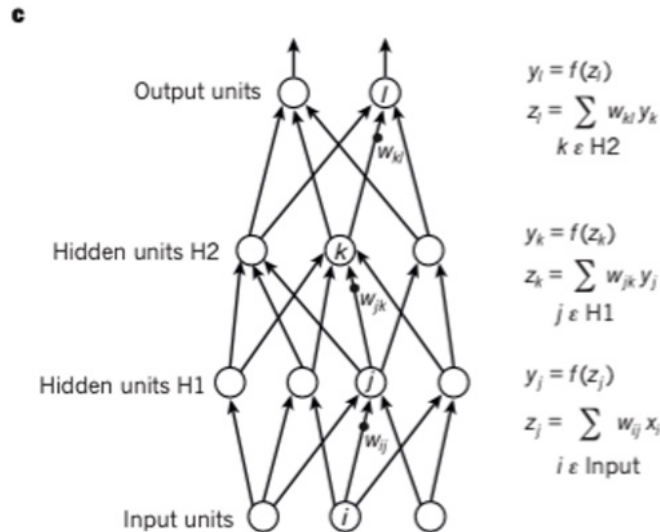
    def forward(self, X):
        #Propagate inputs though network
        self.z2 = np.dot(X, self.W1)
        self.a2 = self.sigmoid(self.z2)
        self.z3 = np.dot(self.a2, self.W2)
        yHat = self.sigmoid(self.z3)
        return yHat

    def sigmoid(self, z):
        #Apply sigmoid activation function to scalar, vector, or matrix
        return 1/(1+np.exp(-z))
```

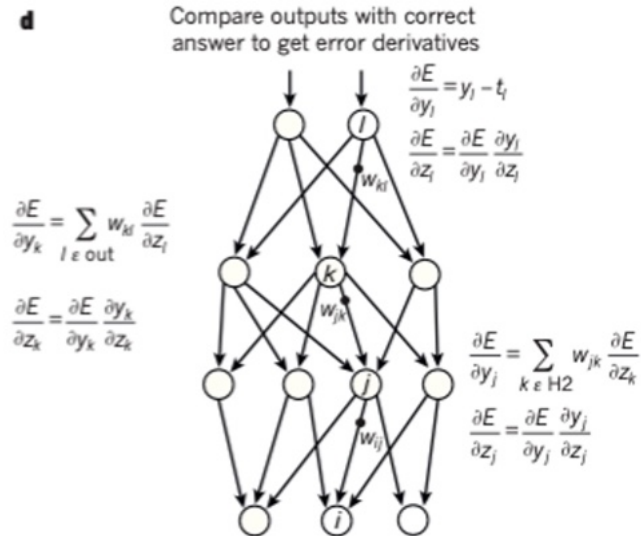


Training with Backpropagation

Forward pass: evaluation



Reverse pass: backpropagation

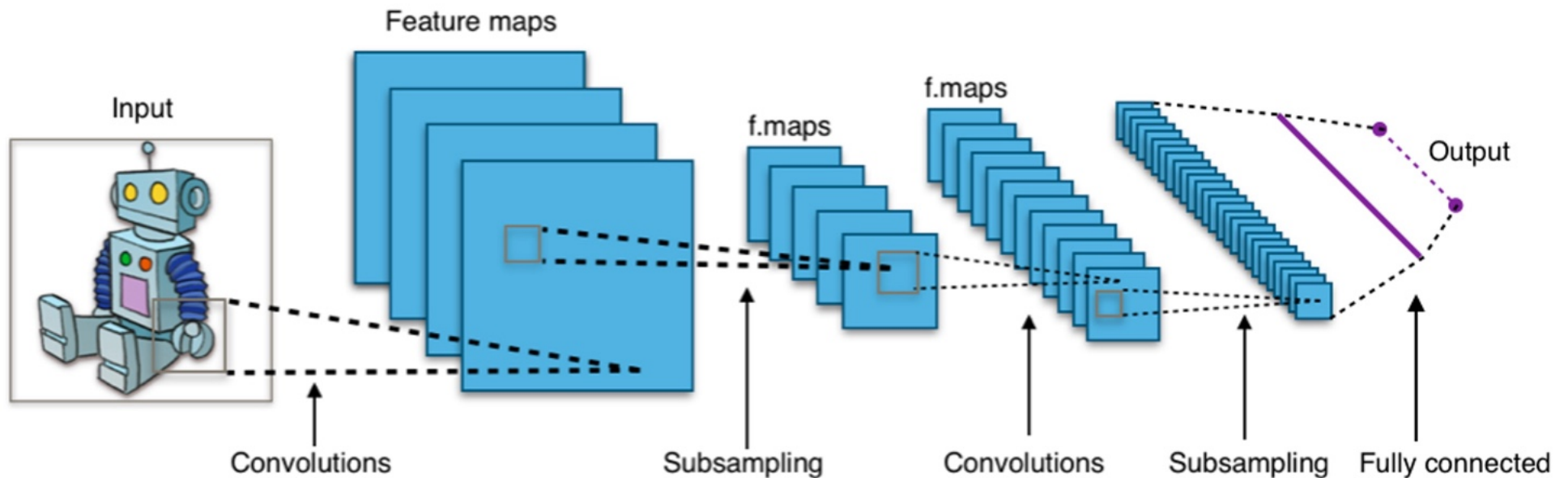


- Calculations along every edge, but the math is generally very basic
 - Addition, multiplication, exponentiation, etc
- Also easy to parallelize (especially with GPUs)

Deep Learning

LeCun, Bengio, Hinton (2020). Nature. doi:10.1038/nature14539

Convolutional Neural Network

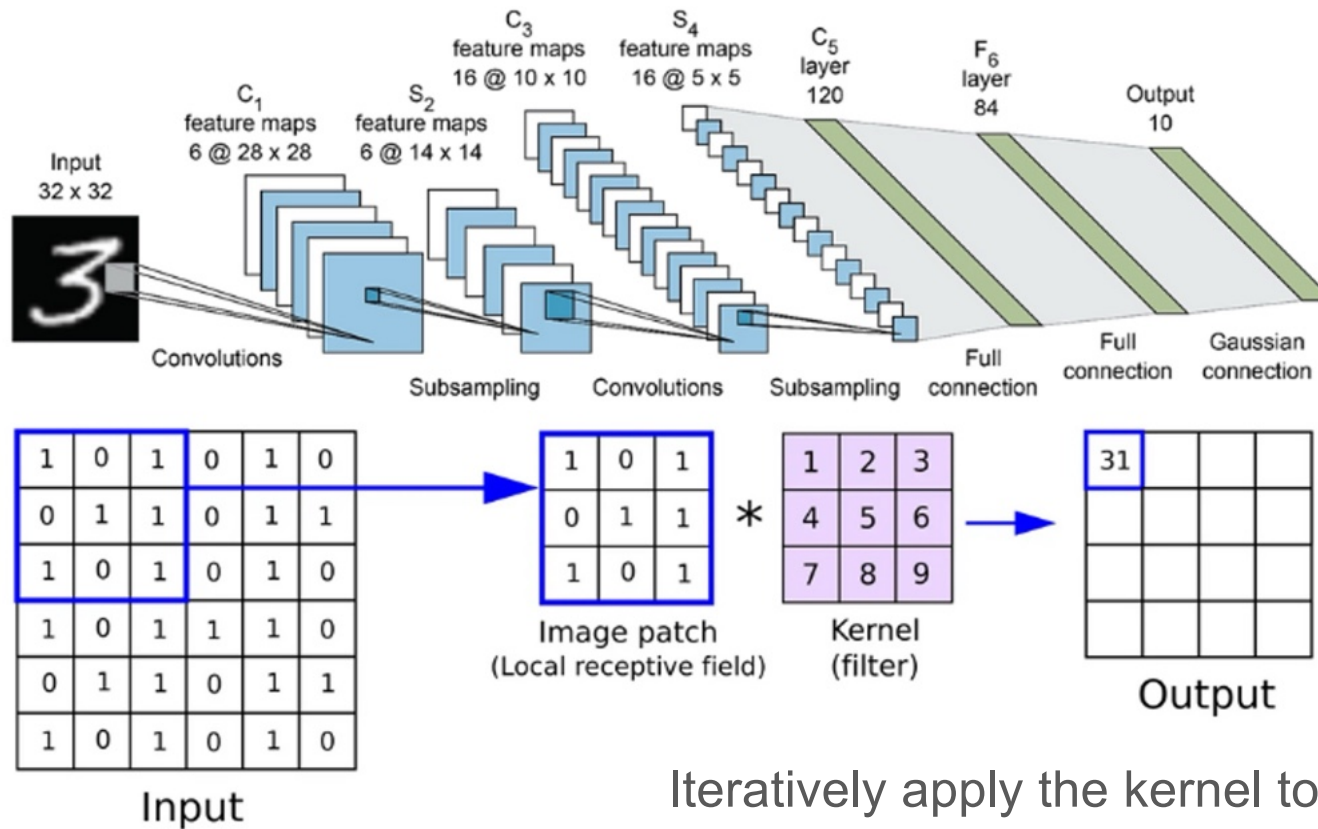


Fully connected neural networks are most expressive but are computationally intensive!

CNNs achieve nearly the same power but with much less compute:

- $O(\text{Nodes} * \text{Depth})$ rather than $O(\text{Nodes} ^ \text{Depth})$

Convolutional Neural Network



- Iteratively apply the kernel to extract features
- Starts from the input image, then creates more abstract derived images

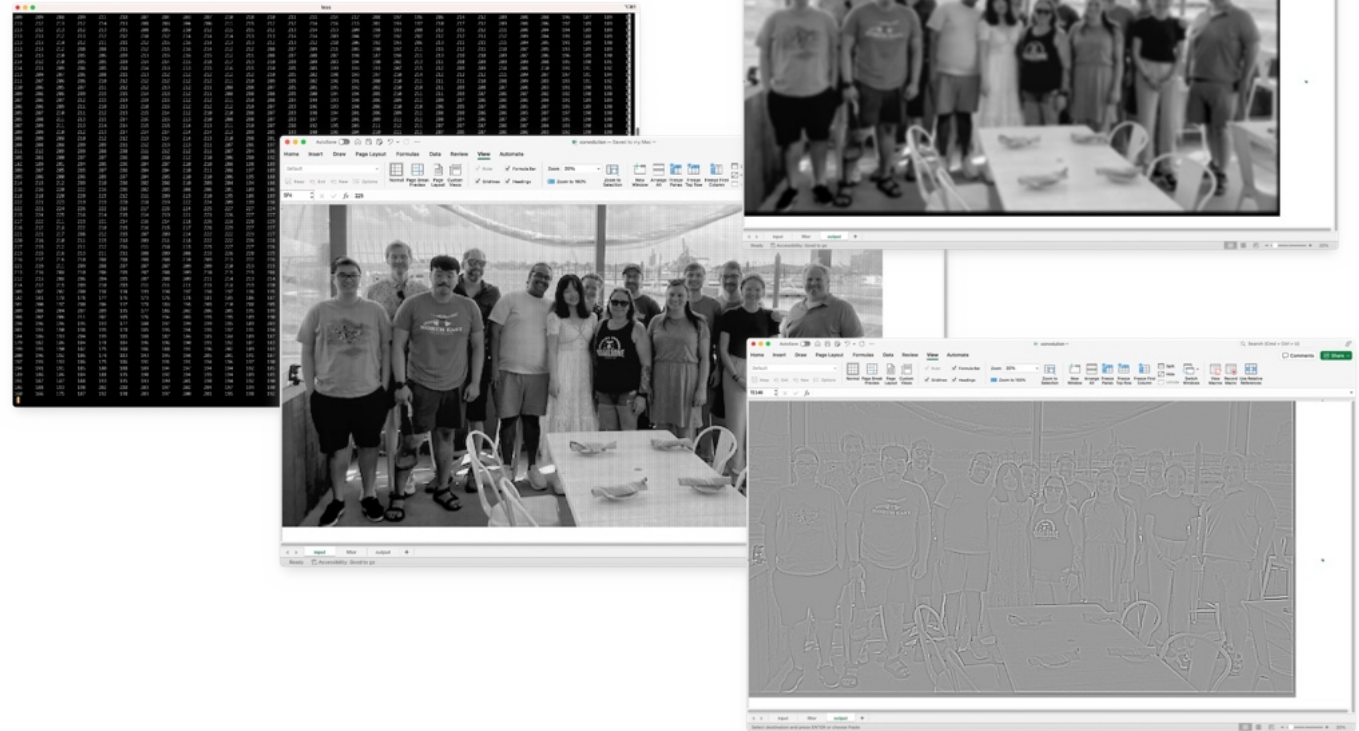
“LeNet” (Lecun et al, 1998)

Kernels

Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 x 3 (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	
Gaussian blur 5 x 5 (approximation)	$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	
Unsharp masking 5 x 5 Based on Gaussian blur with amount as 1 and threshold as 0 (with no image mask)	$\frac{-1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & -476 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	

Kernels

Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 x 3 (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	
Gaussian blur 5 x 5 (approximation)	$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	
Unsharp masking 5 x 5 Based on Gaussian blur with amount as 1 and threshold as 0 (with no image mask)	$\frac{-1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & -476 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	



= input!A1 * filter!\$A\$1 + input!A2 * filter!\$A\$2 + input!A3 * filter!\$A\$3 + input!A4 * filter!\$A\$4 + input!A5 * filter!\$A\$5 +
input!B1 * filter!\$B\$1 + input!B2 * filter!\$B\$2 + input!B3 * filter!\$B\$3 + input!B4 * filter!\$B\$4 + input!B5 * filter!\$B\$5 +
input!C1 * filter!\$C\$1 + input!C2 * filter!\$C\$2 + input!C3 * filter!\$C\$3 + input!C4 * filter!\$C\$4 + input!C5 * filter!\$C\$5 +
input!D1 * filter!\$D\$1 + input!D2 * filter!\$D\$2 + input!D3 * filter!\$D\$3 + input!D4 * filter!\$D\$4 + input!D5 * filter!\$D\$5 +
input!E1 * filter!\$E\$1 + input!E2 * filter!\$E\$2 + input!E3 * filter!\$E\$3 + input!E4 * filter!\$E\$4 + input!E5 * filter!\$E\$5

Outline

1. The Problem
2. pre-AlexNet approaches
3. How do animals & humans approach this?
4. Artificial Neural Networks
5. Convolutional Neural Networks
6. AlexNet
7. Impact

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
krizhev@cs.toronto.edu

Olga Turian
turian@cs.toronto.edu

Geoffrey E. Hinton
hinton@cs.toronto.edu

Abstract

We trained a deep, convolutional neural network to classify the 1 million high-resolution images in the ImageNet ILSVRC2012 contest and achieved the best performance in the contest. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

1 Introduction

Convolutional neural networks have been shown to be effective for image classification. In this paper, we describe a deep, convolutional neural network that was trained on the ImageNet ILSVRC2012 contest. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

1

ImageNet is a large-scale image classification contest. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

2 The Dataset

The ImageNet dataset consists of 1.2 million high-resolution images. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

3 Training on Multiple GPUs

ImageNet is a large-scale image classification contest. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

4 The Architecture

The architecture of the network is described in Figure 1. The network consists of three main parts: a convolutional layer, a pooling layer, and a fully connected layer.

Figure 1: The architecture of the network.

2

5.1.1. Results



Figure 1: The results of the ImageNet ILSVRC2012 contest. The network achieved the best performance in the contest.

5.2. Training on Multiple GPUs

ImageNet is a large-scale image classification contest. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

5.3. The Architecture

The architecture of the network is described in Figure 1. The network consists of three main parts: a convolutional layer, a pooling layer, and a fully connected layer.

3

5.3. Local Response Normalization

Local response normalization is a technique for normalizing the output of a convolutional layer. It is used to improve the performance of the network.

$$L_i = \frac{1}{\sum_{j=1}^n w_j^2} \sum_{j=1}^n w_j^2 x_j$$

The local response normalization is used to improve the performance of the network. It is used to normalize the output of a convolutional layer.

5.4. Out-of-Distribution

The out-of-distribution is a technique for handling images that are not in the training set. It is used to improve the performance of the network.

5.5. Results

The results of the ImageNet ILSVRC2012 contest are shown in Figure 1. The network achieved the best performance in the contest.

5.6. The Architecture

The architecture of the network is described in Figure 1. The network consists of three main parts: a convolutional layer, a pooling layer, and a fully connected layer.

4



Figure 1: The architecture of the ImageNet ILSVRC2012 contest. The network achieved the best performance in the contest.

5.6. Results

The results of the ImageNet ILSVRC2012 contest are shown in Figure 1. The network achieved the best performance in the contest.

5.7. The Architecture

The architecture of the network is described in Figure 1. The network consists of three main parts: a convolutional layer, a pooling layer, and a fully connected layer.

5.8. Results

The results of the ImageNet ILSVRC2012 contest are shown in Figure 1. The network achieved the best performance in the contest.

5

ImageNet is a large-scale image classification contest. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

5. Results

The results of the ImageNet ILSVRC2012 contest are shown in Figure 1. The network achieved the best performance in the contest.

5.6. The Architecture

The architecture of the network is described in Figure 1. The network consists of three main parts: a convolutional layer, a pooling layer, and a fully connected layer.

5.7. Results

The results of the ImageNet ILSVRC2012 contest are shown in Figure 1. The network achieved the best performance in the contest.

5.8. The Architecture

The architecture of the network is described in Figure 1. The network consists of three main parts: a convolutional layer, a pooling layer, and a fully connected layer.

6

ImageNet is a large-scale image classification contest. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

5. Results

The results of the ImageNet ILSVRC2012 contest are shown in Figure 1. The network achieved the best performance in the contest.

5.6. The Architecture

The architecture of the network is described in Figure 1. The network consists of three main parts: a convolutional layer, a pooling layer, and a fully connected layer.

5.7. Results

The results of the ImageNet ILSVRC2012 contest are shown in Figure 1. The network achieved the best performance in the contest.

5.8. The Architecture

The architecture of the network is described in Figure 1. The network consists of three main parts: a convolutional layer, a pooling layer, and a fully connected layer.

7

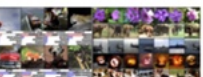


Figure 1: The results of the ImageNet ILSVRC2012 contest. The network achieved the best performance in the contest.

The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

5. Results

The results of the ImageNet ILSVRC2012 contest are shown in Figure 1. The network achieved the best performance in the contest.

5.6. The Architecture

The architecture of the network is described in Figure 1. The network consists of three main parts: a convolutional layer, a pooling layer, and a fully connected layer.

8

ImageNet is a large-scale image classification contest. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days. The network was trained on a GPU with two GeForce GTX 580s, and the training time was 16 days.

5. Results

The results of the ImageNet ILSVRC2012 contest are shown in Figure 1. The network achieved the best performance in the contest.

5.6. The Architecture

The architecture of the network is described in Figure 1. The network consists of three main parts: a convolutional layer, a pooling layer, and a fully connected layer.

5.7. Results

The results of the ImageNet ILSVRC2012 contest are shown in Figure 1. The network achieved the best performance in the contest.

5.8. The Architecture

The architecture of the network is described in Figure 1. The network consists of three main parts: a convolutional layer, a pooling layer, and a fully connected layer.

9

Network Architecture

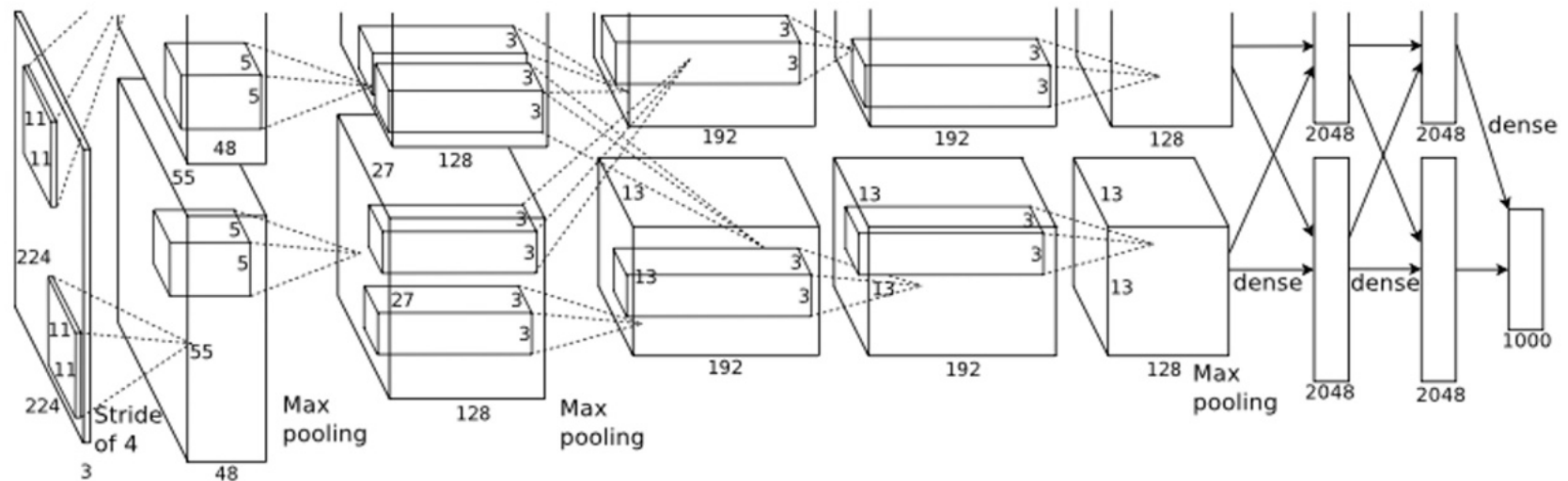
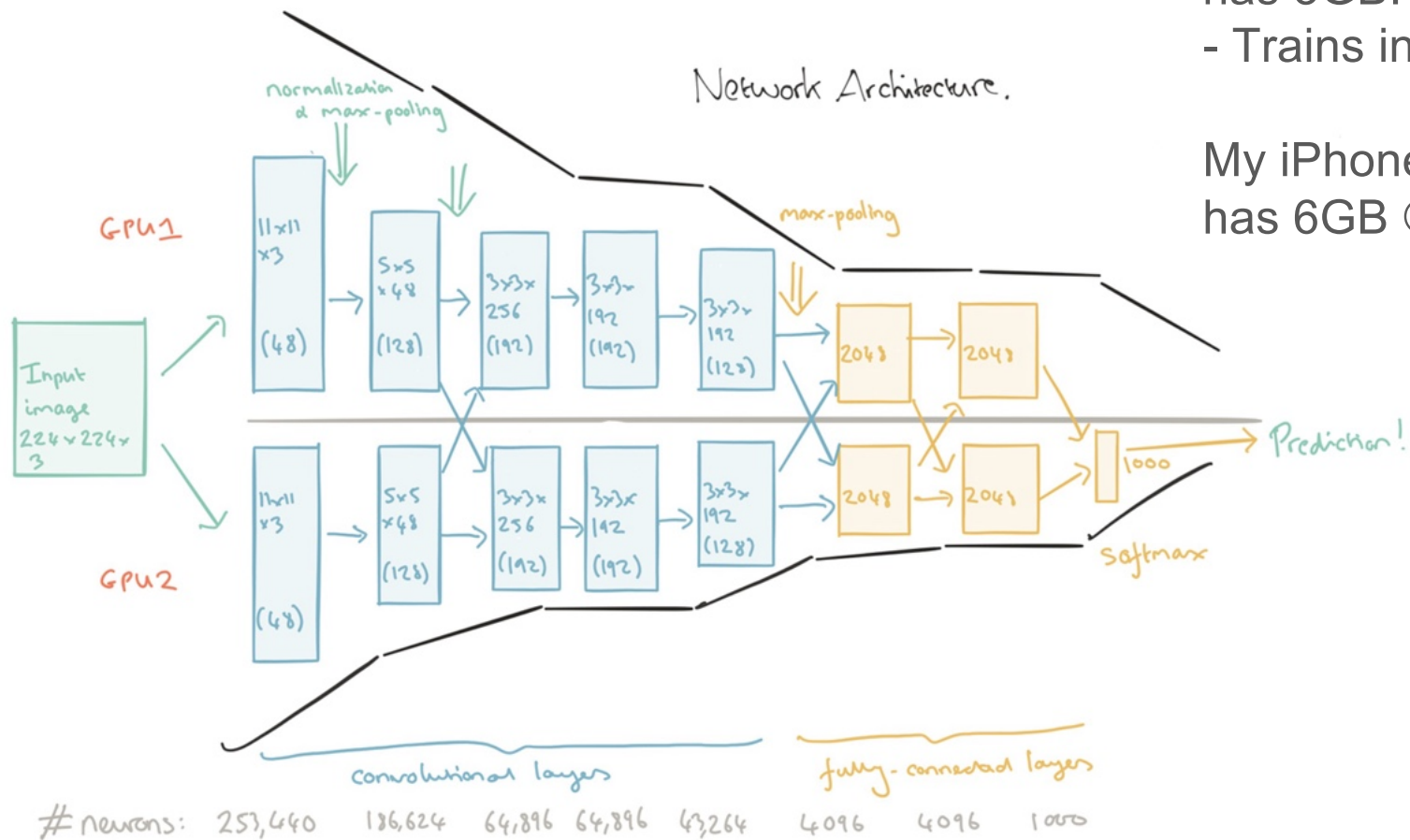


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

Network Architecture (annotated)

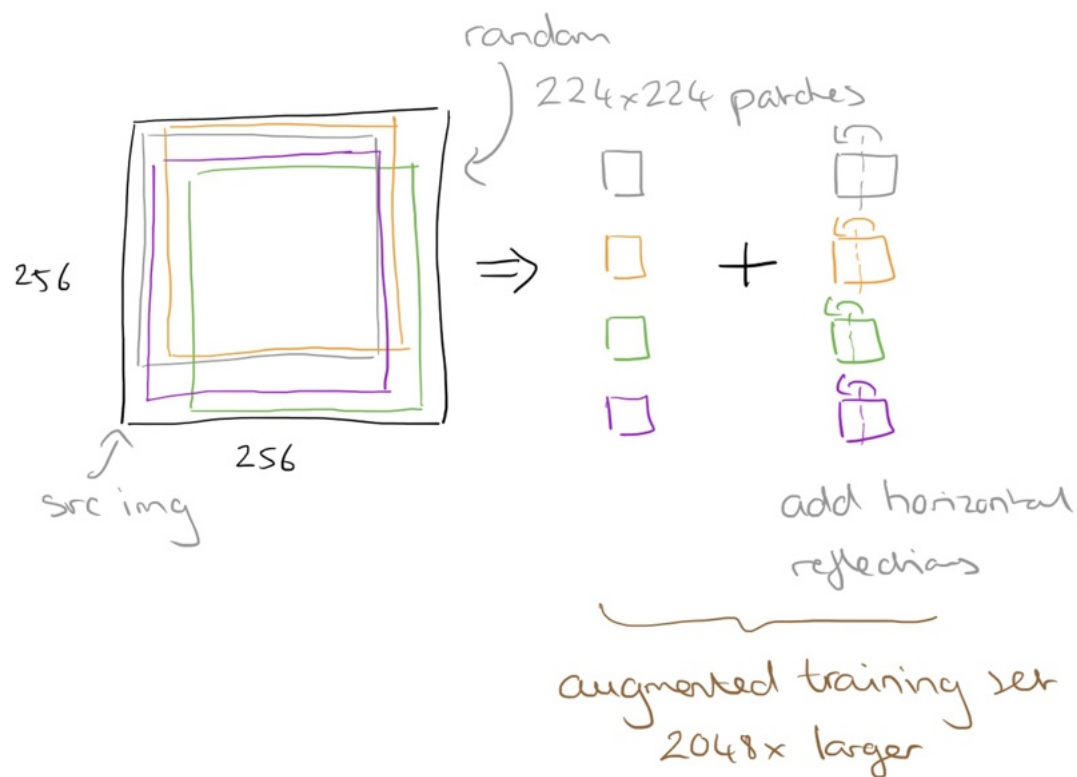
GTX 580 GPU only
has 3GB!
- Trains in ~5 days

My iPhone's GPU
has 6GB 😊



<https://blog.acolyer.org/2016/04/20/imagenet-classification-with-deep-convolutional-neural-networks/>

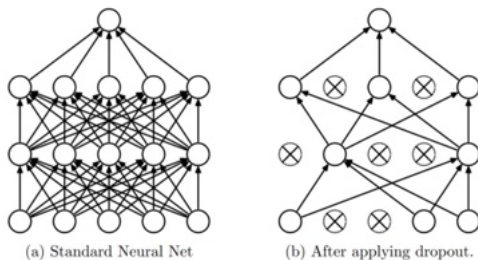
Training data augmentation: prevent overfitting



(See paper for a few other techniques)

Dropout during training

Reducing Overfitting - Dropout



- Output of each hidden neuron is set to zero with probability 0.5
- Learning more robust features
- Doubles the number of iterations required to converge
- Applied in the first two fully connected layers

The recently-introduced technique, called “dropout”, consists of setting to zero the output of each hidden neuron with probability 0.5. The neurons which are “dropped out” in this way do not contribute to the forward pass and do not participate in back-propagation. **So every time an input is presented, the neural network samples a different architecture, but all these architectures share weights.** This technique reduces complex co-adaptations of neurons, since a neuron cannot rely on the presence of particular other neurons. ***It is, therefore, forced to learn more robust features that are useful in conjunction with many different random subsets of the other neurons.***

[N. Srivastava et al., Dropout: A Simple Way to Prevent Neural Networks from Overfitting, 2014]

Training process

Stochastic Gradient Descent

- Training process

- Minimizing the cross-entropy loss function:

$$L(w) = \sum_{i=1}^N \sum_{c=1}^{1000} -y_{ic} \log f_c(x_i) + \epsilon \|w\|_2^2$$

indicator that example i has label c

predicted probability of class c for image x

- SGD with a batch size of 128
- Learning rate initialized at 0.01; divided by 10 if validation error rate stopped improving
- Update rule for weight w:

$$v_{i+1} := 0.9 * v_i - 0.0005 * \epsilon * w_i - \epsilon * \left\langle \frac{\partial L}{\partial w} \Big|_{w_i}, D_i \right\rangle$$

momentum

weight decay

learning rate

$$w_{i+1} := w_i + v_{i+1}$$

Gradient of Loss

- ~ 90 cycles → five to six days on two NVIDIA GTX 580 3GB GPUs

https://cvml.ista.ac.at/courses/DLWT_W17/material/AlexNet.pdf

Learned Convolutional Kernels

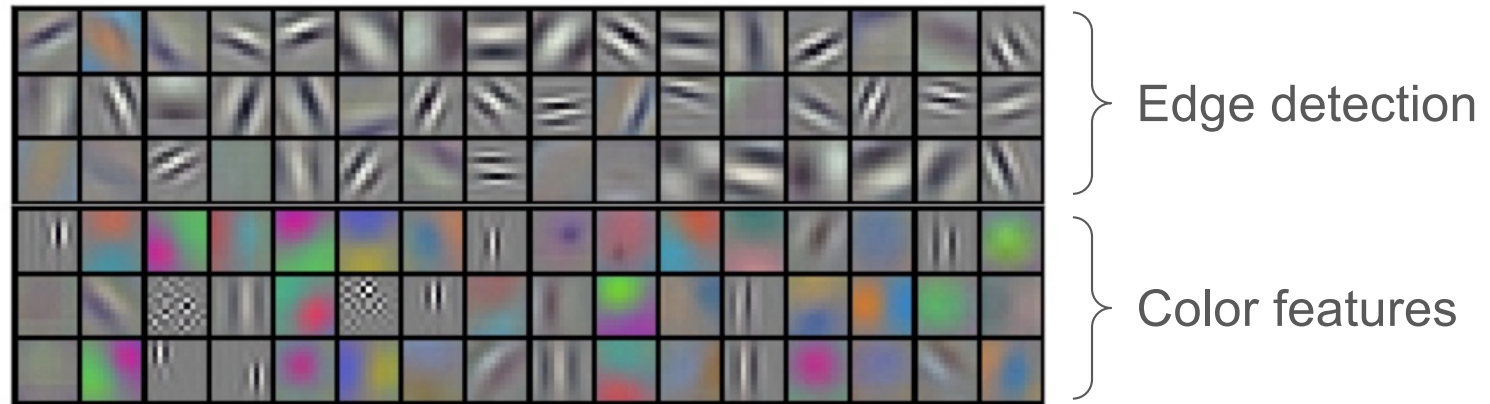
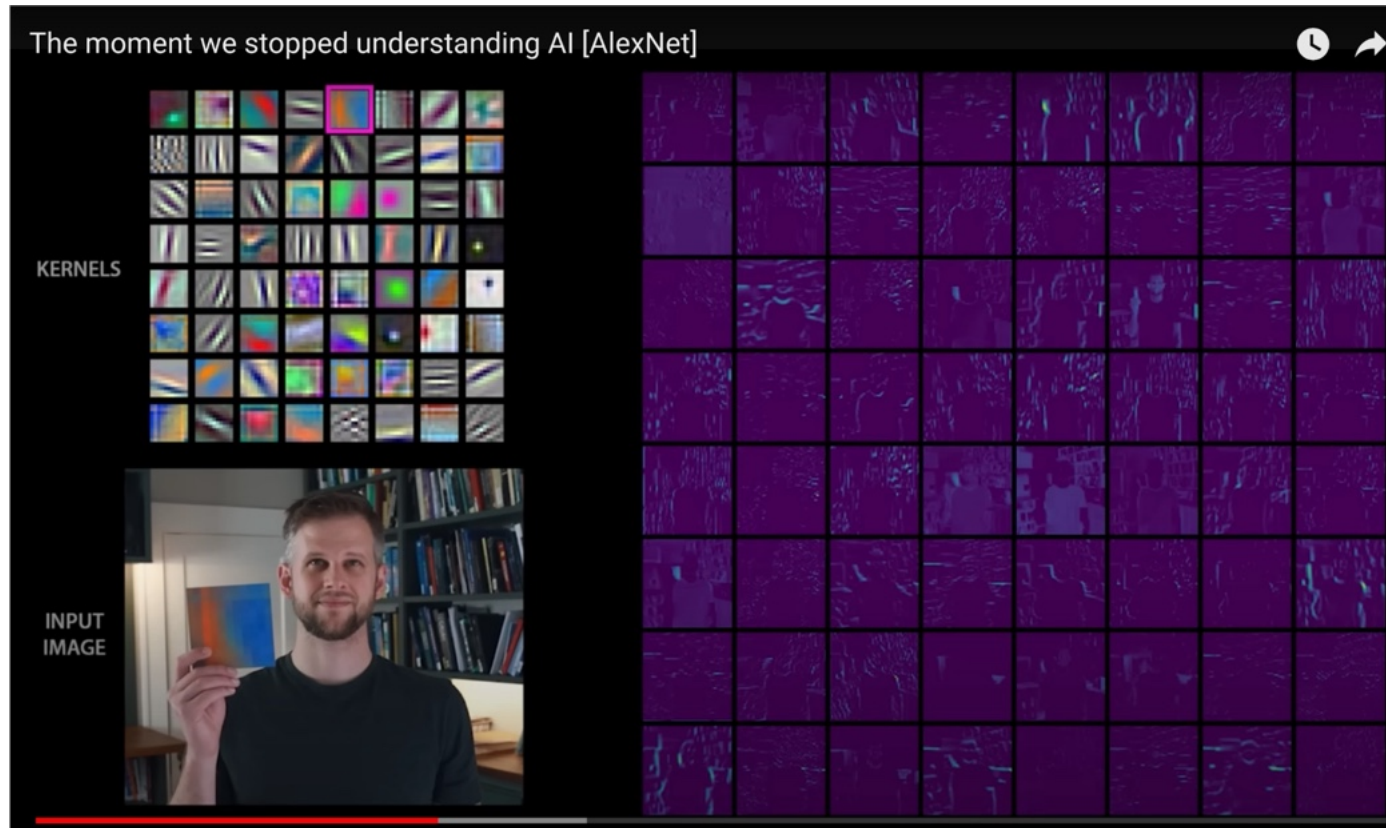


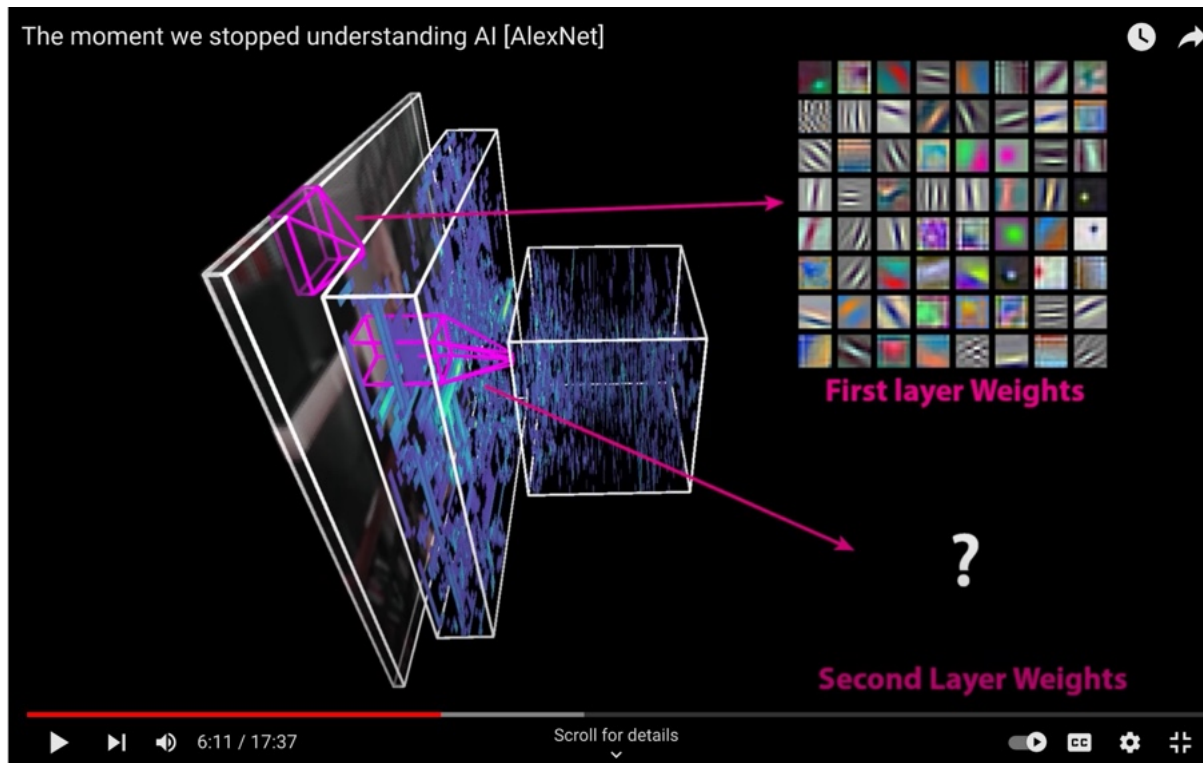
Figure 3: 96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images. The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU 2. See Section 6.1 for details.

From 1 image to 96 images: Activation Maps



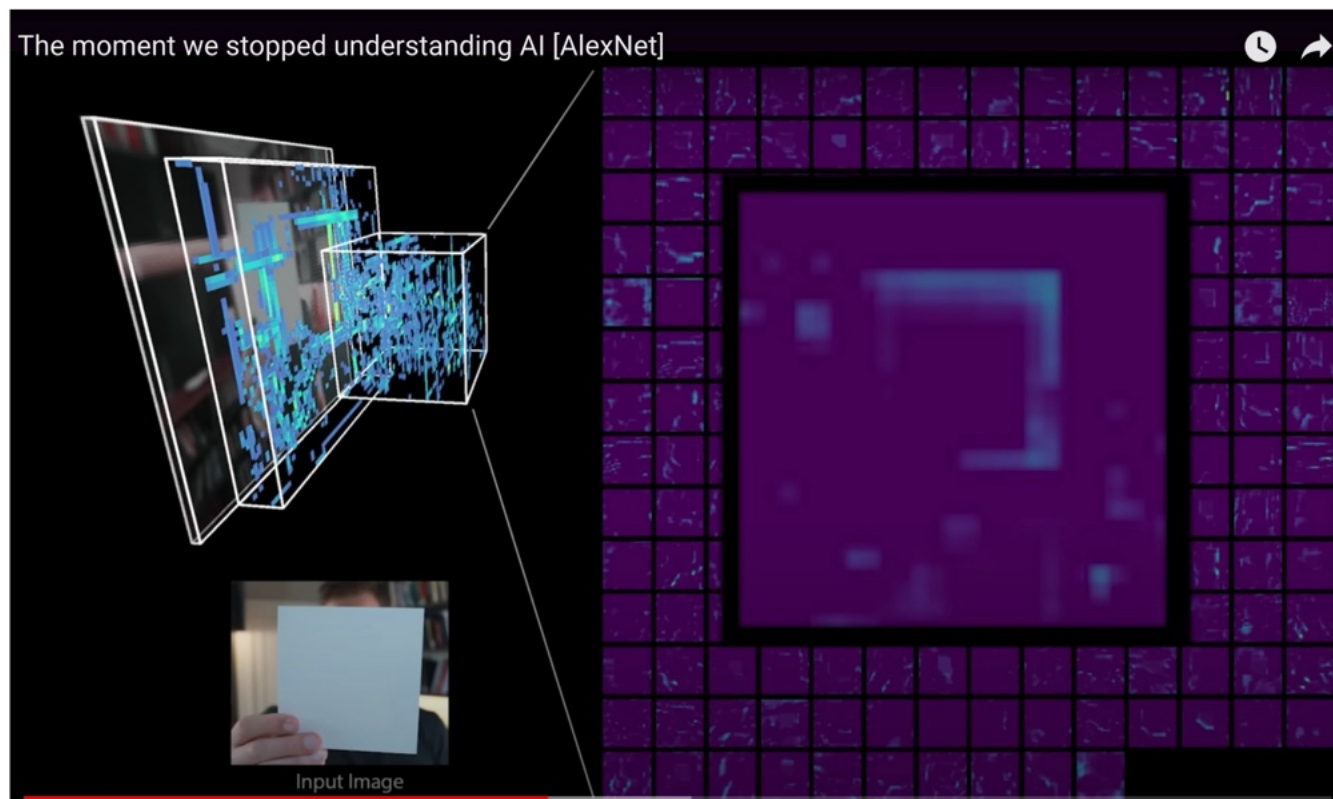
<https://www.youtube.com/watch?v=UZDiGooFs54>

Repeat convolution for deeper layers
But hard to visualize: 1 image becomes a stack of 96



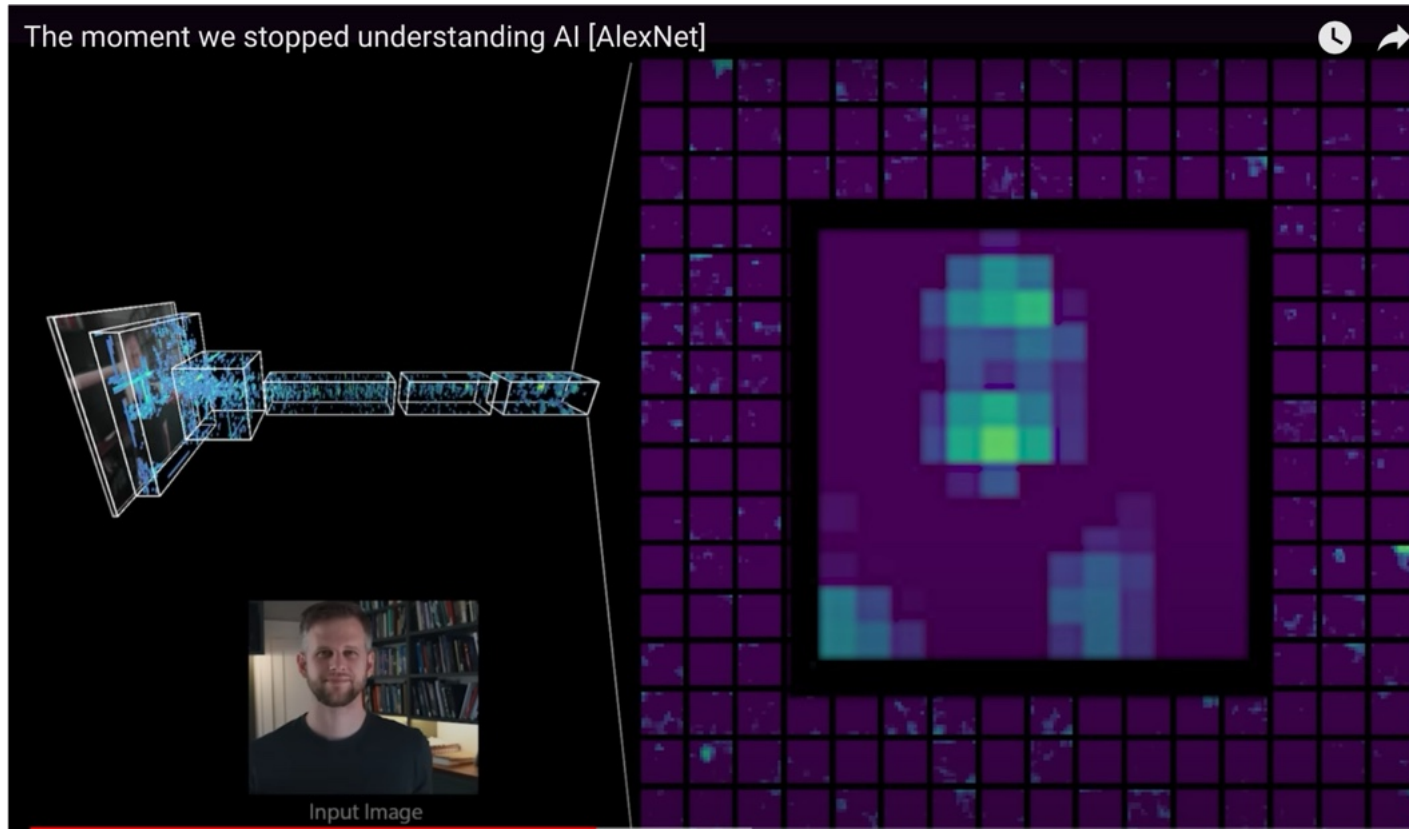
<https://www.youtube.com/watch?v=UZDiGooFs54>

Testing images shows the deeper layers are learning more complicated features



<https://www.youtube.com/watch?v=UZDiGooFs54>

Deeper layers recognize very complex features: faces!

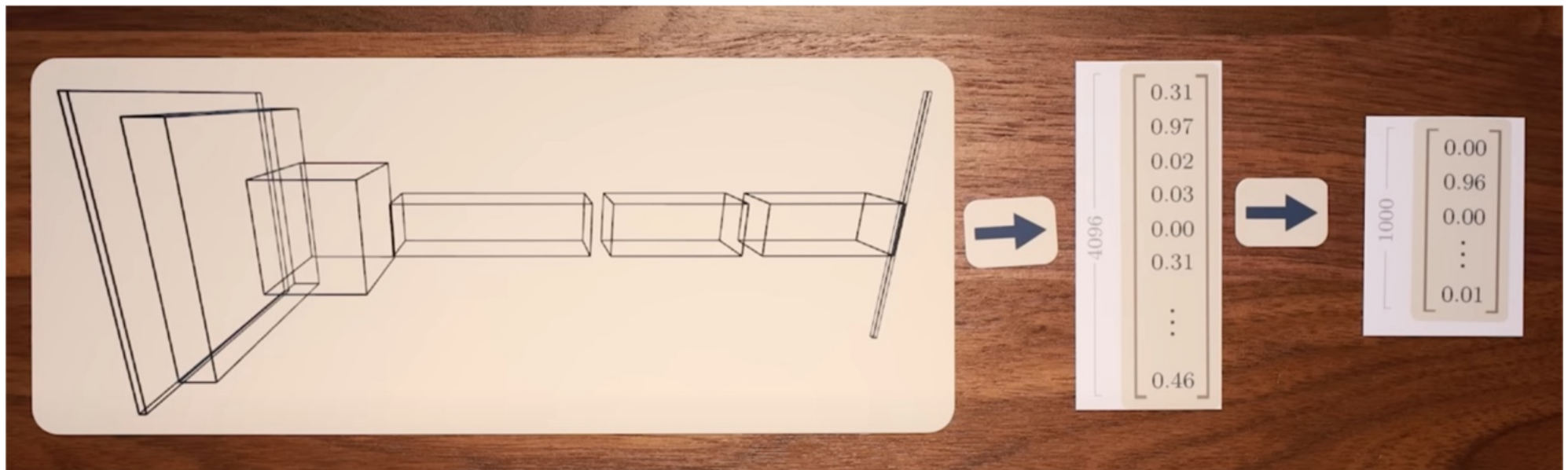


<https://www.youtube.com/watch?v=UZDiGooFs54>

Final two layers are special:

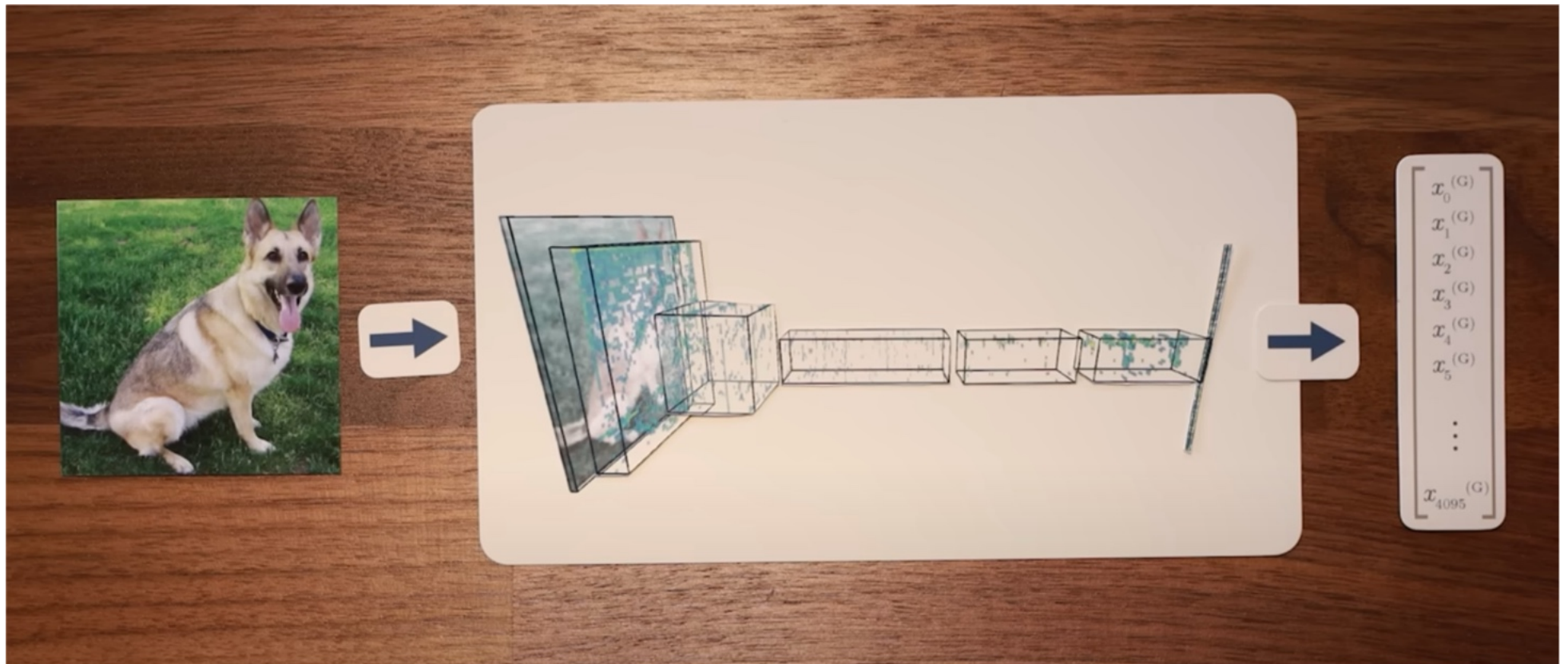
Last layer $[1 \times 1000]$: Probability of the image in class i

Second last $[1 \times 4096]$: Projection of image into 4096-dim space



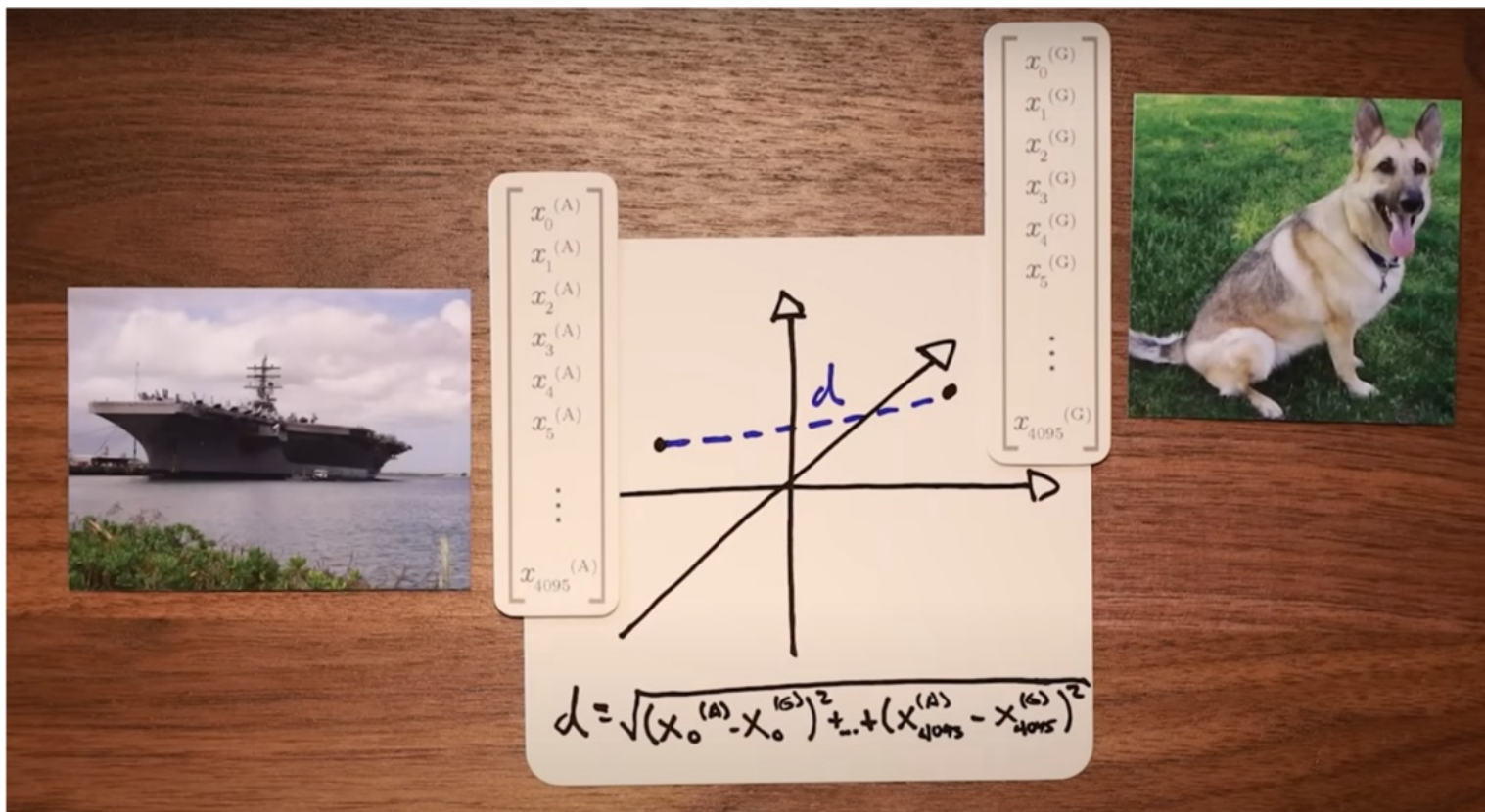
<https://www.youtube.com/watch?v=UZDiGooFs54>

Second to last forms an embedding of the input image



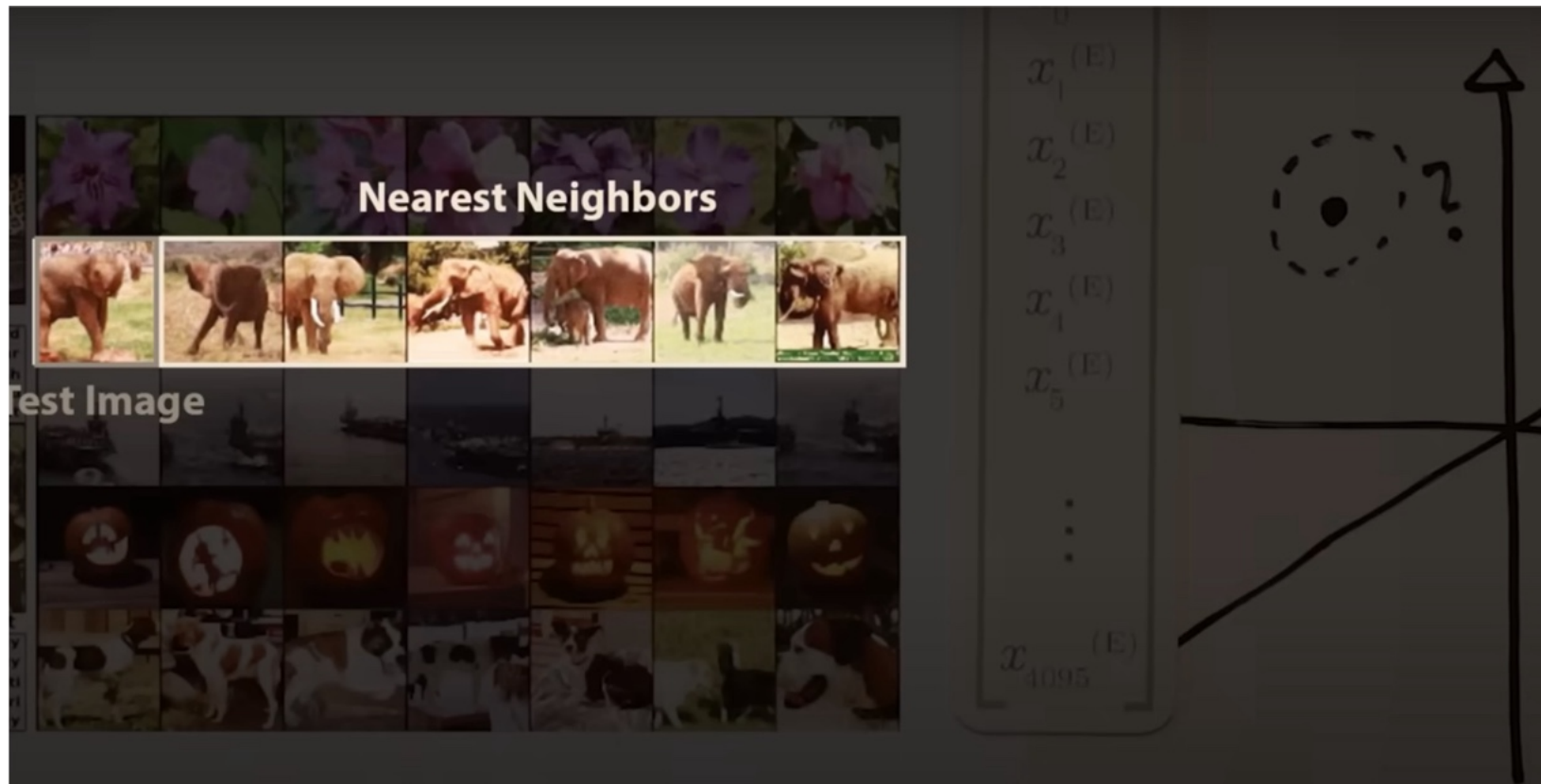
<https://www.youtube.com/watch?v=UZDiGooFs54>

Each image creates a unique vector
Compute the distance between vectors in 4096-dim space



<https://www.youtube.com/watch?v=UZDiGooFs54>

Nearest neighbors in 4096-dim space are semantically related!
Robust to object orientation, size, color, etc



<https://www.youtube.com/watch?v=UZDiGooFs54>

Activation Atlas: tSNE plot of images embeddings



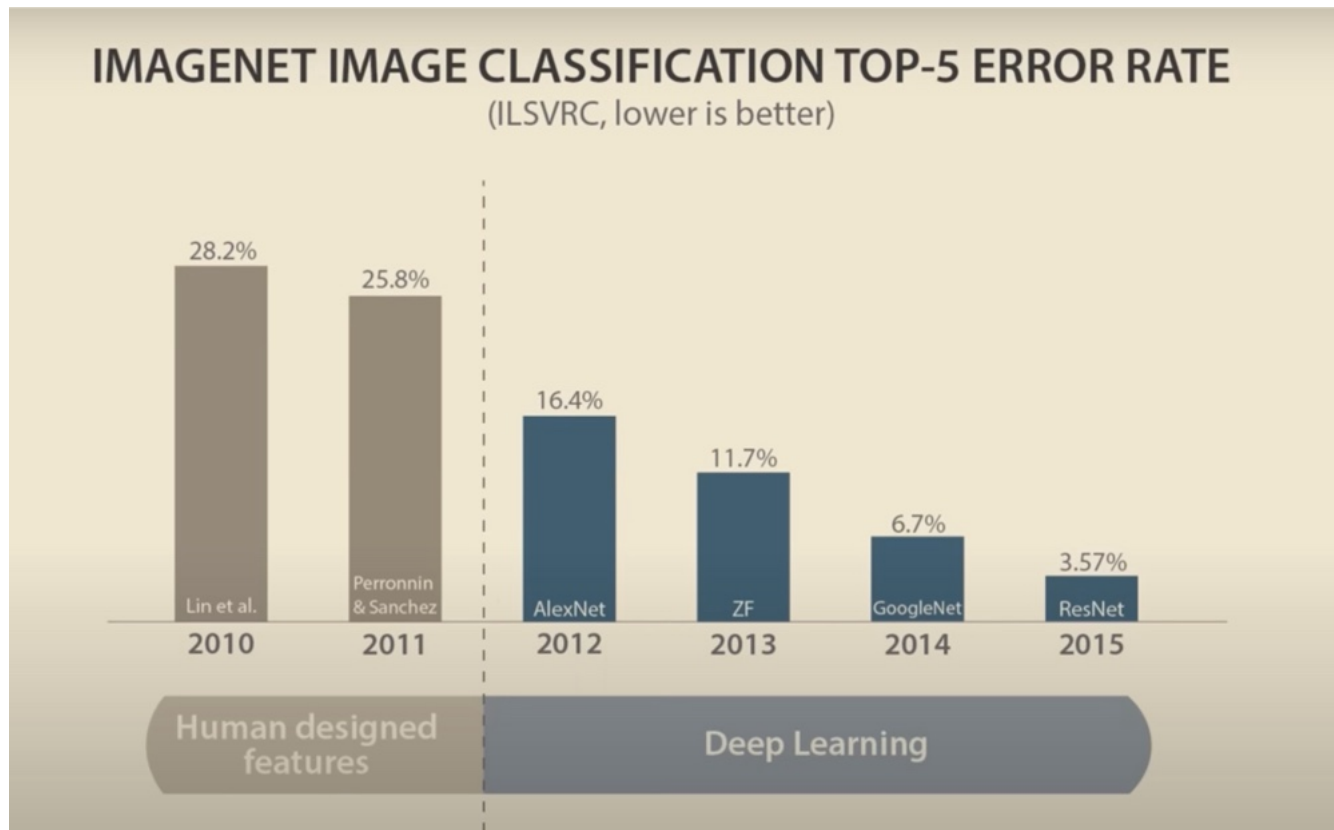
<https://www.youtube.com/watch?v=UZDiGooFs54>

Activation Atlas: tSNE plot of images embeddings



<https://www.youtube.com/watch?v=UZDiGooFs54>

Deep Learning Success!



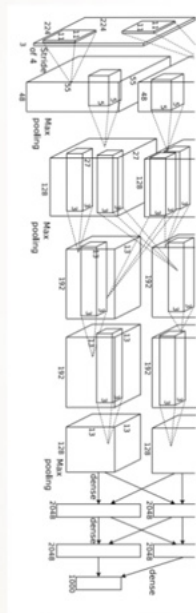
*“Our results show that a large, deep convolutional neural network is capable of achieving record-breaking results on a highly challenging dataset using purely supervised learning. It is notable that our network’s performance degrades if a single convolutional layer is removed. For example, removing any of the middle layers results in a loss of about 2% for the top-1 performance of the network. **So the depth really is important for achieving our results.**”*

<https://www.youtube.com/watch?v=UZDiGooFs54>

Outline

1. The Problem
2. pre-AlexNet approaches
3. How do animals & humans approach this?
4. Artificial Neural Networks
5. Convolutional Neural Networks
6. AlexNet
7. Impact

“AlexNet”



[Krizhevsky et al. NIPS 2012]

“GoogLeNet”



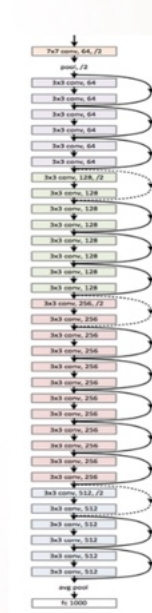
[Szegedy et al. CVPR 2015]

“VGG Net”



[Simonyan & Zisserman, ICLR 2015]

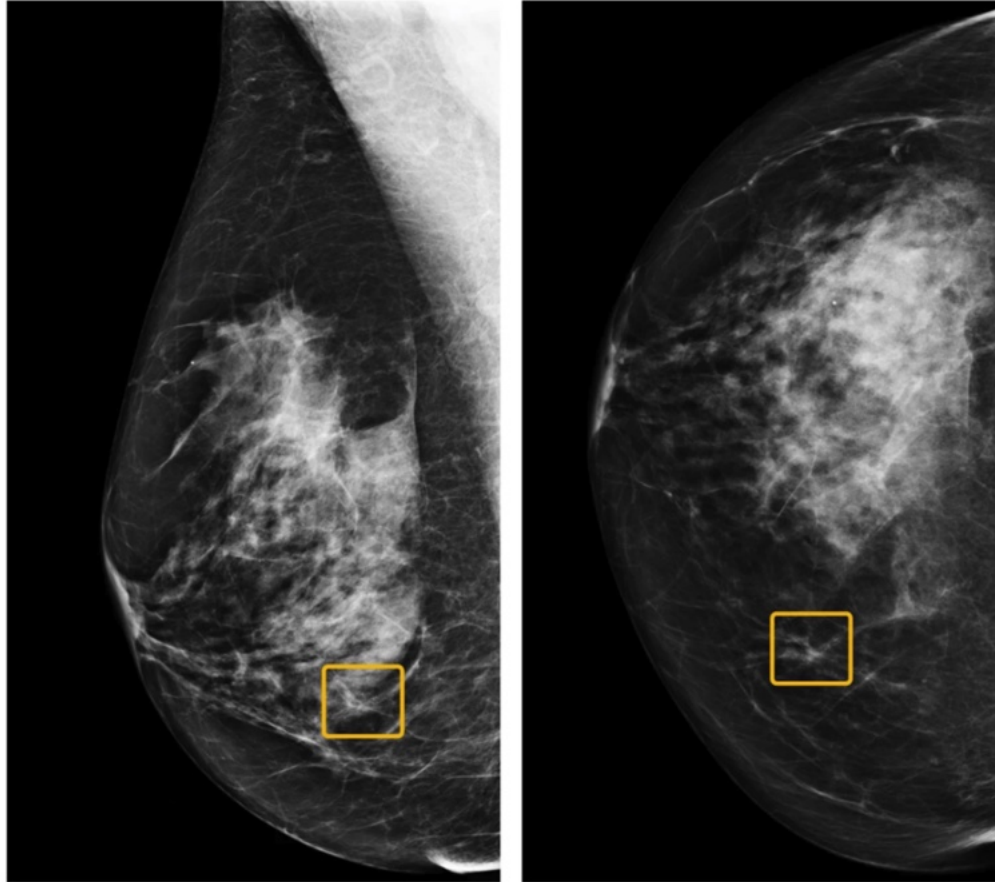
“ResNet”



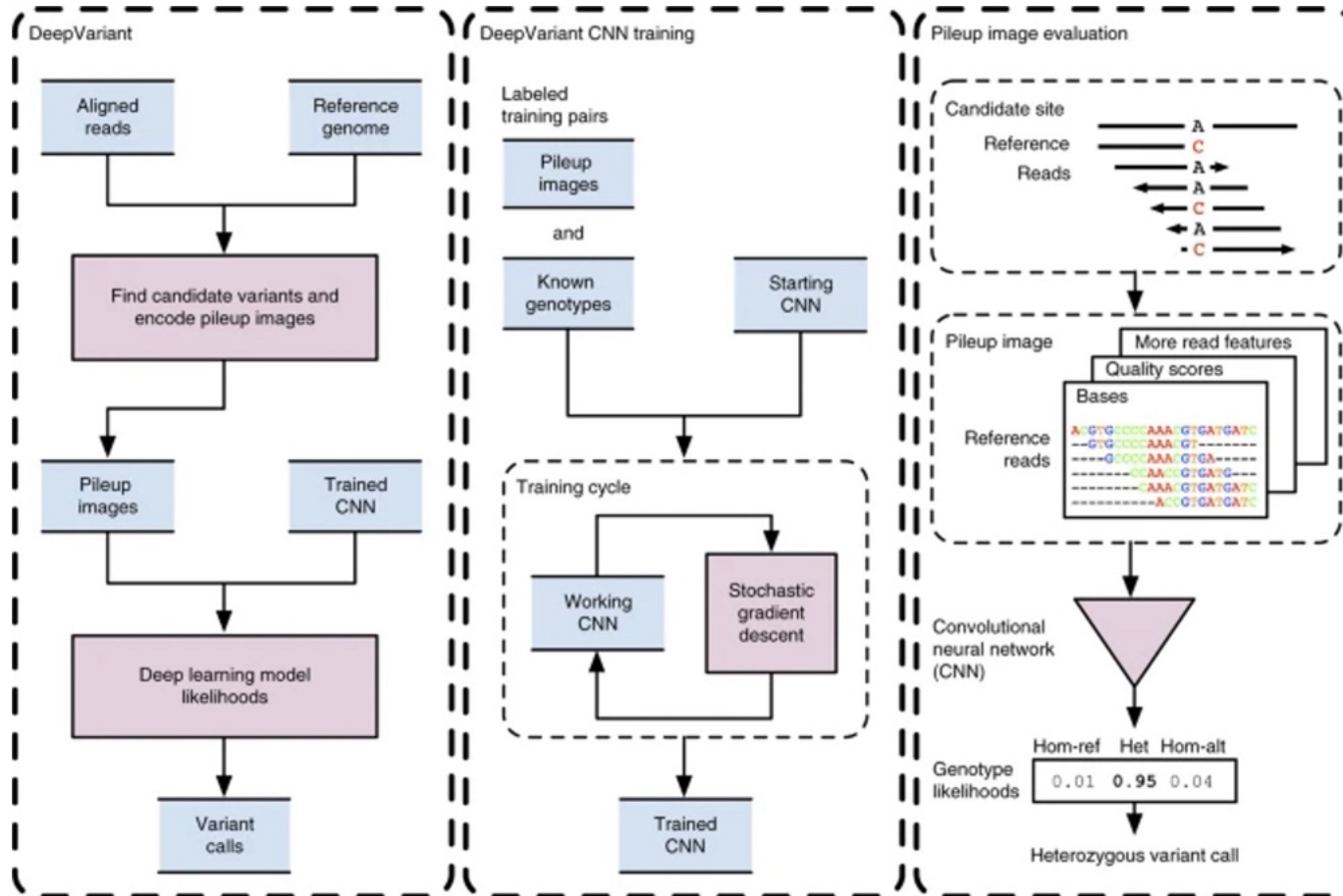
[He et al. CVPR 2016]

imagenet_ilsrvrc2017_v1.0

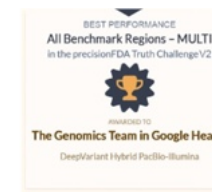
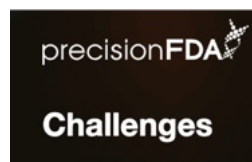
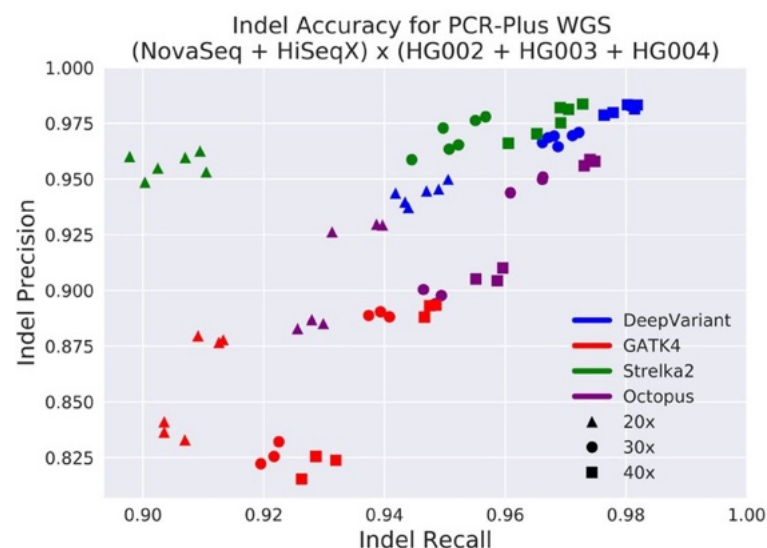
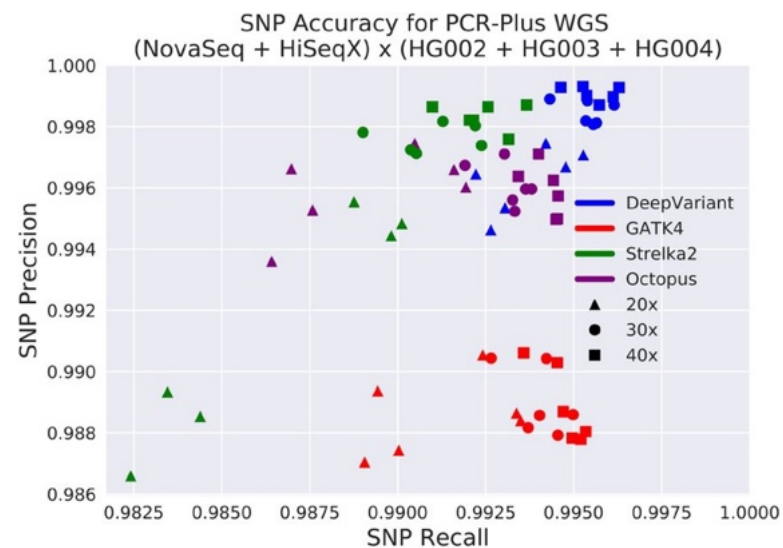
a



International evaluation of an AI system for breast cancer screening
McKinney et al (2020) Nature. <https://doi.org/10.1038/s41586-019-1799-6>



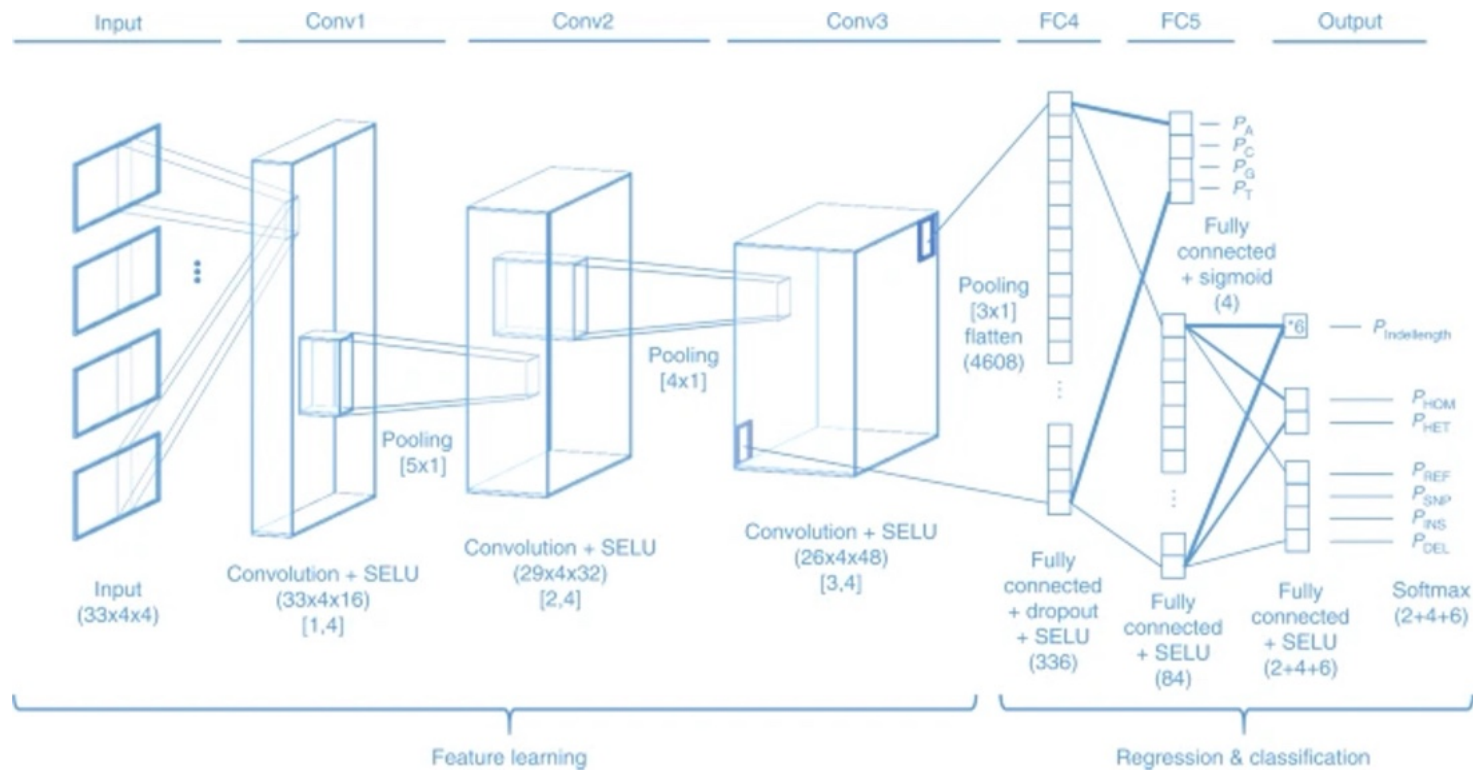
A universal SNP and small-indel variant caller using deep neural networks
 Poplin et al (2018) Nature Biotechnology. <https://doi.org/10.1038/nbt.4235>



An Extensive Sequence Dataset of Gold-Standard Samples for Benchmarking and Development

Baid et al (2020) bioRxiv

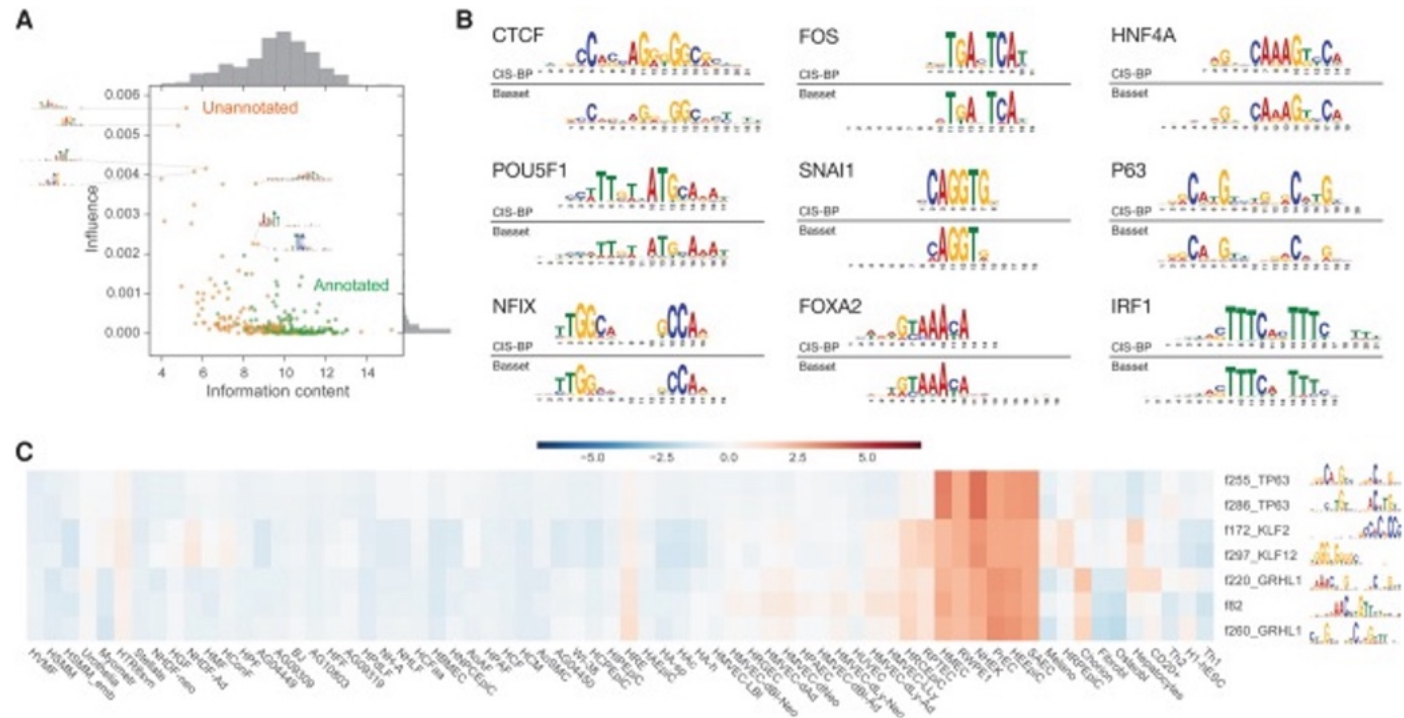
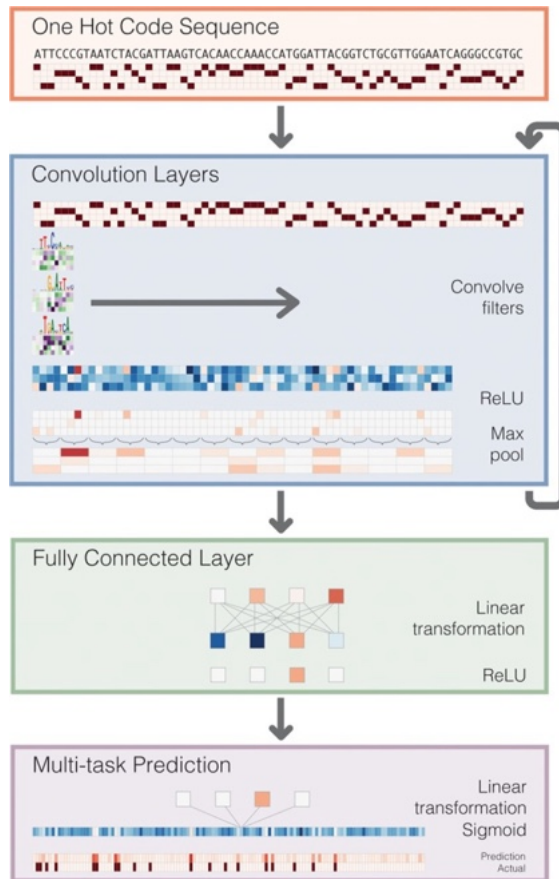
<https://www.biorxiv.org/content/10.1101/2020.12.11.422022v1.full>



A multi-task convolutional deep neural network for variant calling in single molecule sequencing

Luo, Sedlazeck, Lam, Schatz (2019) Nature Communication.

<https://doi.org/10.1038/s41467-019-09025-z>



Basset: learning the regulatory code of the accessible genome with deep convolutional neural networks
 Kelly et al (2016) Genome Research. doi: 10.1101/gr.200535.115

ANNs are “Universal Approximators”

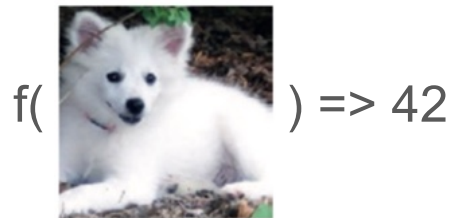
$$f(\text{img}) \Rightarrow 42$$
A small, fluffy white dog, likely a Samoyed, is lying down on a dark, textured surface. The dog is looking towards the camera with its mouth slightly open, showing its teeth. The background is dark and out of focus.

$224 \times 224 = 50k$ pixels

$50k \text{ pixels} \times 3 \text{ bytes (RGB)}$

838B possible inputs

ANNs are “Universal Approximators”



$224 \times 224 = 50k$ pixels

$50k \text{ pixels} \times 3 \text{ bytes (RGB)}$
838B possible inputs

ConvnetJS demo: toy 2d classification with 2-layer neural network

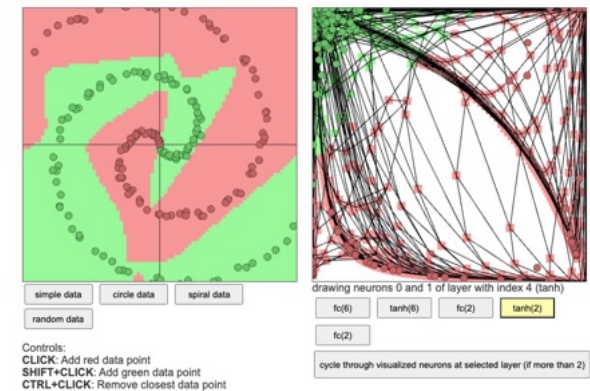
The simulation below shows a toy binary problem with a few data points of class 0 (red) and 1 (green). The network is set up as:

```
layer_defs = [];  
layer_defs.push({type:'input', out_sz:1, out_sy:1, out_depth:2});  
layer_defs.push({type:'fc', num_neurons:6, activation: 'tanh'});  
layer_defs.push({type:'fc', num_neurons:2, activation: 'tanh'});  
layer_defs.push({type:'softmax', num_classes:2});  
net = new convnetjs.Net();  
net.makeLayers(layer_defs);  
trainer = new convnetjs.SGDTrainer(net, {learning_rate:0.01, momentum:0.1, batch_size:10, l2_decay:0.001});
```

change network

Feel free to change this, the text area above gets eval()'d when you hit the button and the network gets reloaded. Every 10th of a second, all points are fed to the network multiple times through the trainer class to train the network. The resulting predictions of the network are then "painted" under the data points to show you the generalization.

On the right we visualize the transformed representation of all grid points in the original space and the data, for a given layer and only for 2 neurons at a time. The number in the bracket shows the total number of neurons at that level of representation. If the number is more than 2, you will only see the two visualized but you can cycle through all of them with the cycle button.



<https://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>



Figure 5: Adversarial examples generated for AlexNet [9].(Left) is a correctly predicted sample, (center) difference between correct image, and image predicted incorrectly magnified by 10x (values shifted by 128 and clamped), (right) adversarial example. All images in the right column are predicted to be an “ostrich, *Struthio camelus*”. Average distortion based on 64 examples is 0.006508. Please refer to <http://goo.gl/huaGPb> for full resolution images. The examples are strictly randomly chosen. There is not any postselection involved.

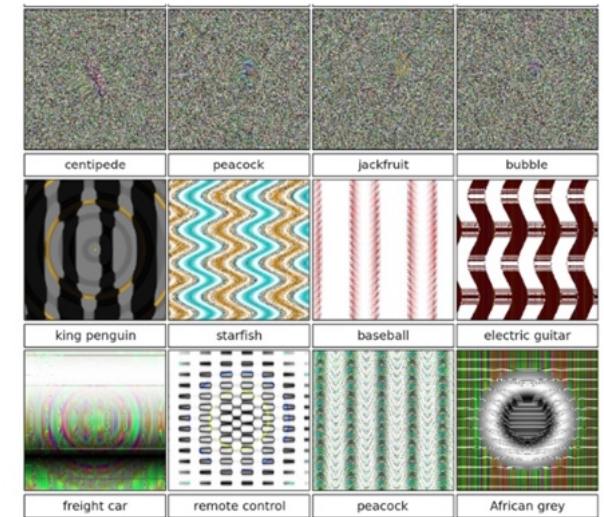


Figure 1. Evolved images that are unrecognizable to humans, but that state-of-the-art DNNs trained on ImageNet believe with $\geq 99.6\%$ certainty to be a familiar object. This result highlights differences between how DNNs and humans recognize objects. Images are either directly (*top*) or indirectly (*bottom*) encoded.

Intriguing properties of neural networks

Szegedy et al (2013) arXiv:1312.6199

Deep Neural Networks are Easily Fooled: High Confidence Predictions for Unrecognizable Images

Nguyen et al (2015) arXiv:1412.1897

YouTube player interface showing a video titled "Why Deep Learning Works Unreasonably Well [How Models Learn Part 3]" by Welch Labs. The video features a 3D visualization of a neural network structure with colorful, translucent planes and a central vertical column of nodes. The text "make sense of everything that's happening" is overlaid on the visualization.

Why Deep Learning Works Unreasonably Well [How Models Learn Part 3]

Welch Labs
796K subscribers

297K views · 1 month ago
Take your personal data back with Incogni! Use code WELCHLABS and get 60% off an annual plan: <http://incogni.com/welchlabs> ...more

16K likes · Share · Ask · Download

Ask & learn
Learn by asking questions, practicing and getting guidance

Related videos:

- Python Data Science for Beginners
- lofi hip hop radio
- Markov Chains
- The Strange Math That Predicts (Almost) Anything
- CATHY ENGELBERT Gets EXPOSED LYING About CAITLIN...
- The moment we stopped understanding AI [AlexNet]
- Python: The Documentary | An origin story

Welch Labs: Why Deep Learning Works Unreasonably Well
<https://www.youtube.com/watch?v=qx7hirqgfuU>

Reflections

- **CNNs are incredibly powerful for image recognition (and similar tasks)**
 - Heavily inspired by how the human brain processes visual information from the real world
 - Start with the most basic feature kernels and then iterate into higher level concepts leveraging their spatial proximity
 - Ultimately projects images into abstract semantic embeddings
- **Certain problems are a great fit for CNNs:**
 - ImageNet: Lots of test data, nicely annotated into 1000 categories
 - DeepVariant/Clairvoyante: Lots of test data; One network architecture made it the worlds best variant caller for all types of long read data: PacBio, ONT, 10x
- **Deeper network architectures generally perform better**
 - Largely constrained by GPU RAM & Cores

“Images” in Bioinformatics

- **DNA sequence: one-hot encode**
 - Are there other encoding schemes? (modDotPlot)
 - Predict features (genes, repeat classes); Predict species (kraken)
- **Coverage tracks:**
 - Predict: SNVs (DeepVariant), CNVs, SVs (Cue)
 - RNAseq, ChIPseq (overcome error, allele specific)
- **Functional Genomics**
 - Inputs
 - Expression from multiple tissues (GTEx) from one patient
 - Multiomics: Variants + expression + methylation
 - Predict phenotype: Gender (easy), cell types (easy), age (pretty easy)
 - Phenopredict: <https://academic.oup.com/nar/article/46/9/e54/4920847>
 - Disease outcome: Needs the right training data, right tissues
- Multiple alignments, position weight matrix
 - Predict conserved regions, predict binding sites, etc

	 LeNet-5	 AlexNet	 GPT-3	 GPT-4
YEAR	1998	2012	2020	2023
TRAINING DATA	· MNIST Dataset · 60k training examples · 10 classes	· ImageNet Dataset (ILSVRC) · 1.2M training examples · 1000 classes	· Common Crawl, WebText, Wikipedia, others · ~500B training tokens · ~100k unique tokens	· ~13T training tokens*
TRAINING COMPUTE	· Pentium II CPU · ~0.27 GFLOPs	· Dual Nvidia GTX 580 · 3162 GFLOPs	· 10,000 Nvidia V100 GPUs · 1+ ExaFLOPs	· 25,000 Nvidia A100s GPUs* · ~4+ ExaFLOPs
ALGORITHM	· ~60k Parameters · 5 Layers · Sigmoid Activation Function	· ~60M Parameters · 8 Layers · ReLU Activation Function · Dropout	· 175B Parameters · 96 Layers · Transformers	· 1T+ Parameters* · *120 Layers · Transformers

<https://www.youtube.com/watch?v=UZDiGooFs54>



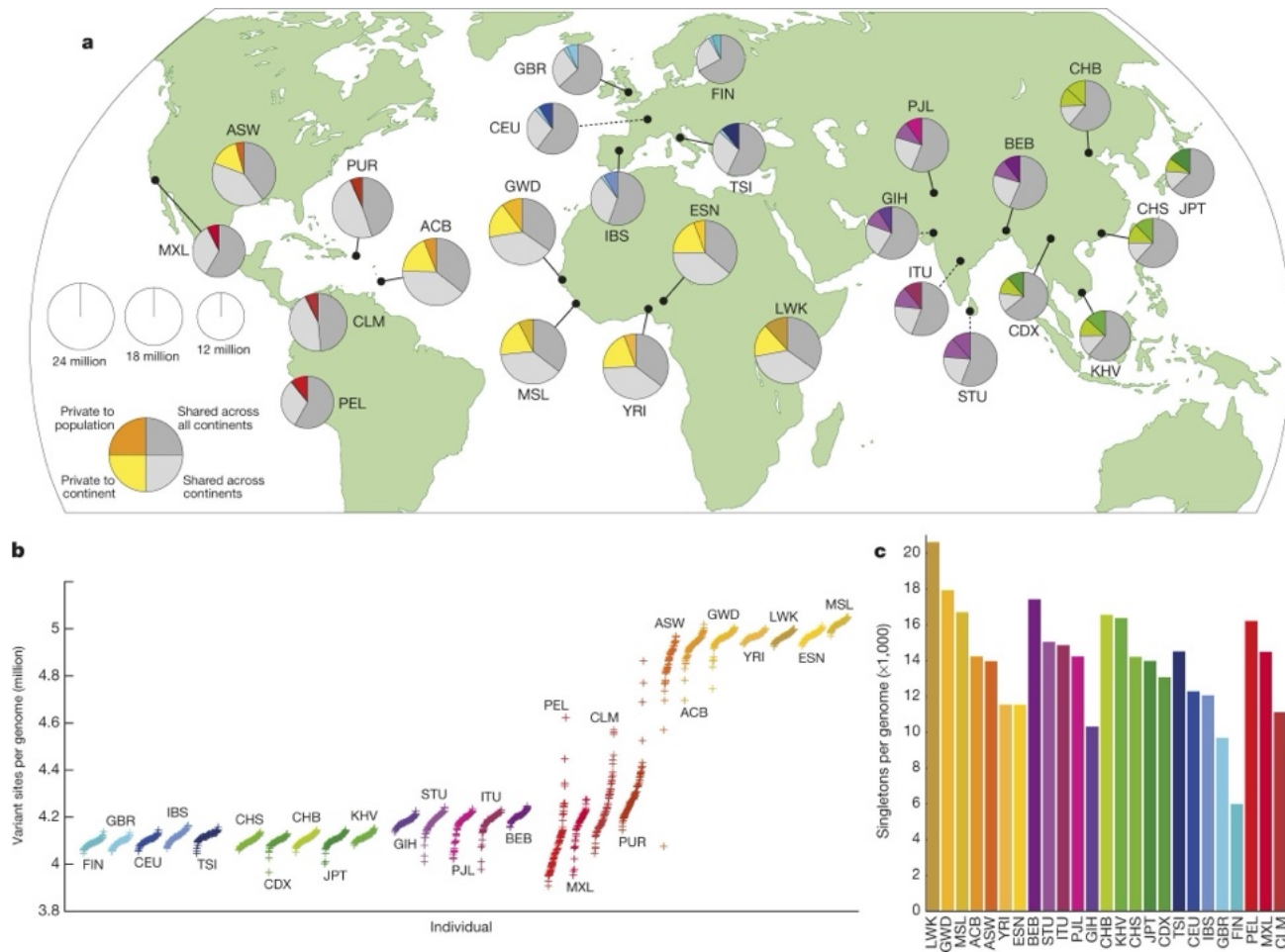
Part II: (Healthy) Humans

A global reference for human genetic variation

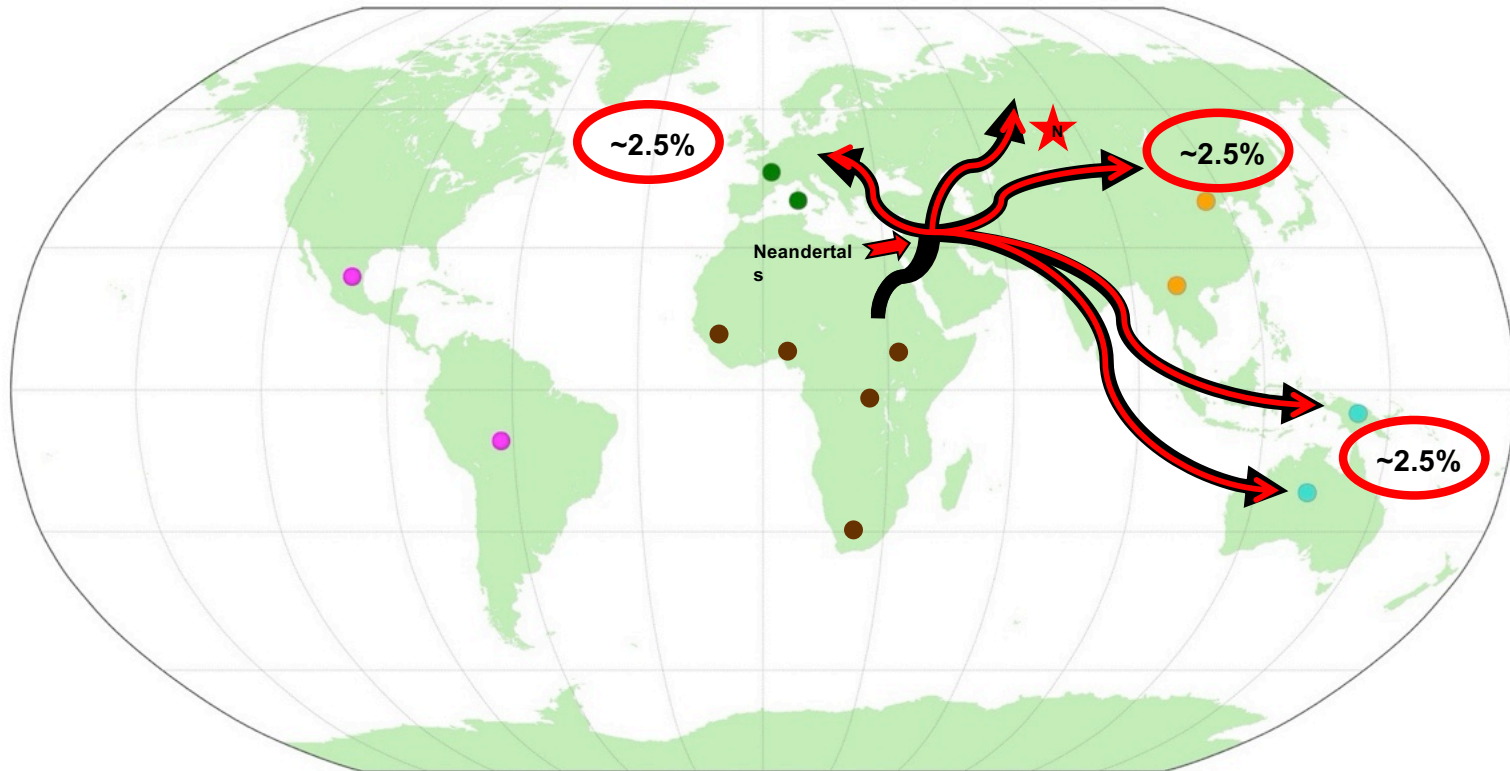
The 1000 Genomes Project Consortium*

The 1000 Genomes Project set out to provide a comprehensive description of common human genetic variation by applying whole-genome sequencing to a diverse set of individuals from multiple populations. Here we report completion of the project, having reconstructed the genomes of 2,504 individuals from 26 populations using a combination of low-coverage whole-genome sequencing, deep exome sequencing, and dense microarray genotyping. We characterized a broad spectrum of genetic variation, in total over 88 million variants (84.7 million single nucleotide polymorphisms (SNPs), 3.6 million short insertions/deletions (indels), and 60,000 structural variants), all phased onto high-quality haplotypes. This resource includes >99% of SNP variants with a frequency of >1% for a variety of ancestries. We describe the distribution of genetic variation across the global sample, and discuss the implications for common disease studies.

1000 Genomes Populations

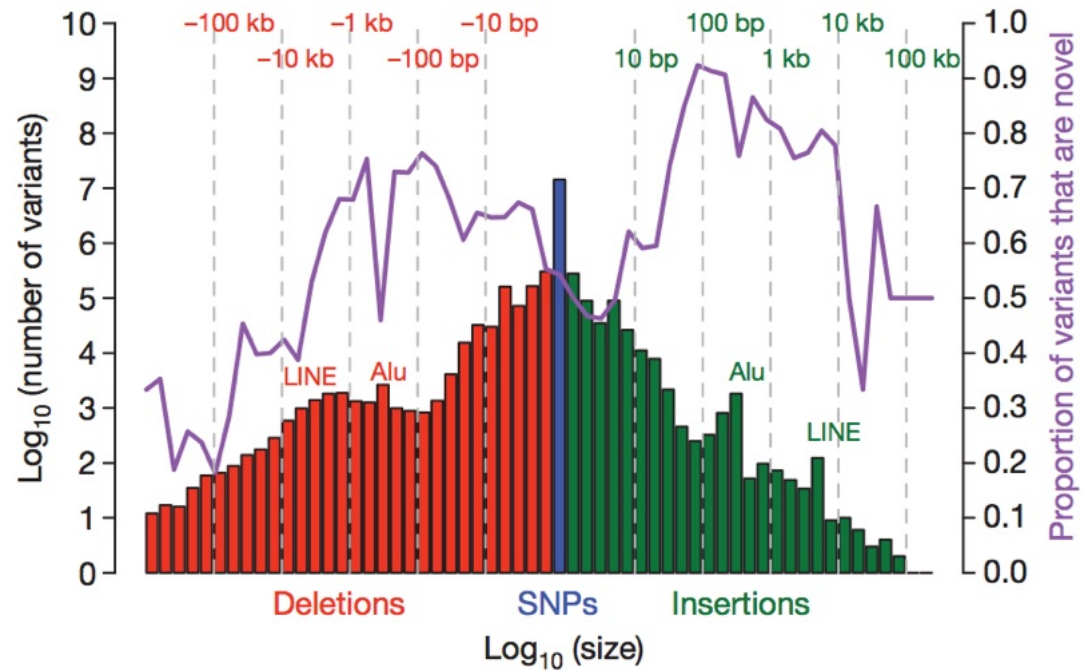


Human Migration Patterns



As modern humans migrated out of Africa, they apparently interbred with Neanderthals so we see their alleles across the rest of the world and carry about 2.5% of their genome with us!

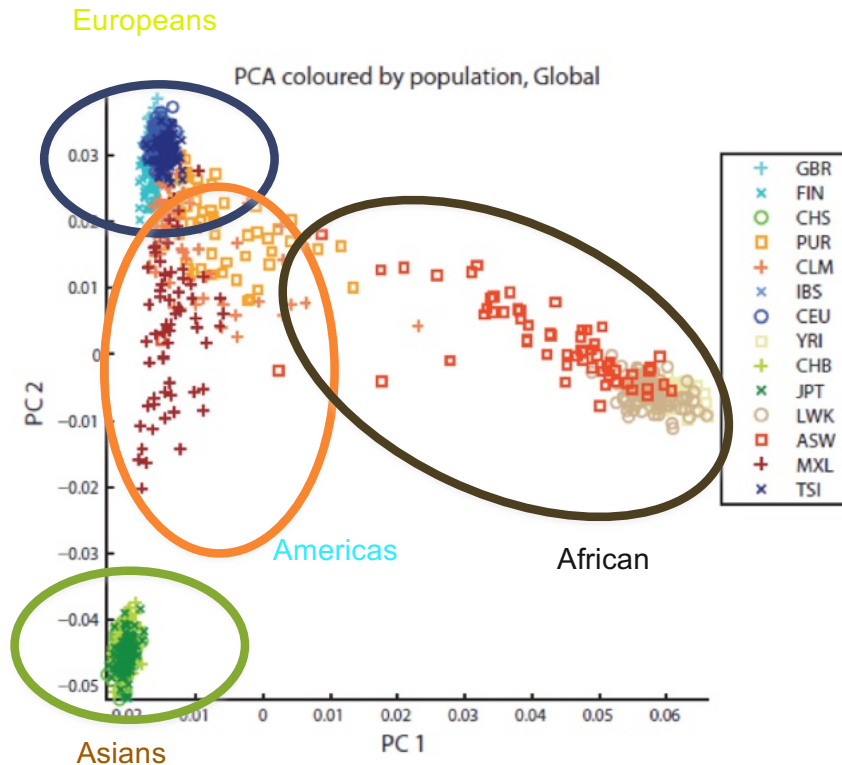
Human Mutation Types



- Mutations follows a “log-normal” frequency distribution
 - Most mutations are SNPs followed by small indels followed by larger events

A map of human genome variation from population-scale sequencing
1000 genomes project (2010) *Nature*. doi:10.1038/nature09534

Variation across populations

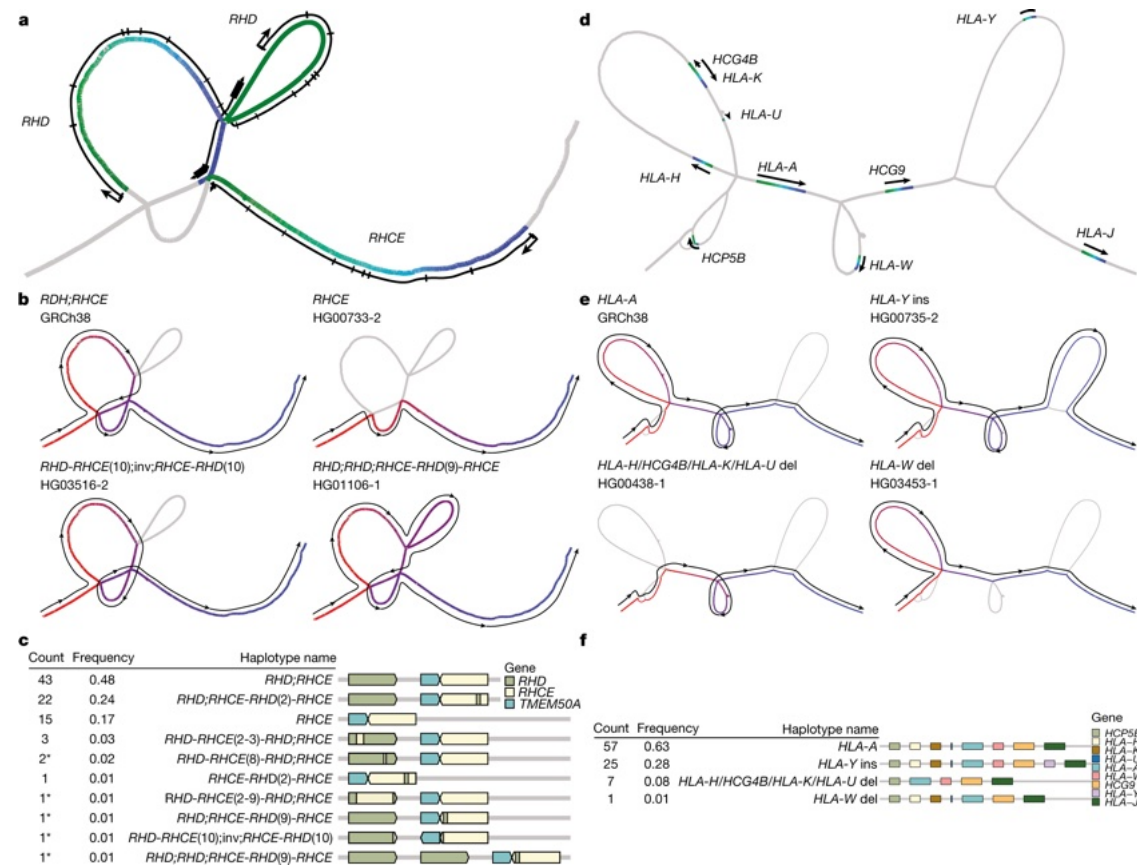


LEVEL	POP_PAIR	# of Highly differentiated SNPs	% in transcribed regions*
AFR	ASW-LWK	258	46.8
AFR	LWK-YRI	251	50.2
AFR	ASW-YRI	213	45.8
ASN	CHS-JPT	275	48.1
ASN	CHB-JPT	176	43.7
ASN	CHB-CHS	79	38.7
EUR	FIN-TSI	343	42.6
EUR	CEU-FIN	201	40.7
EUR	FIN-GBR	197	43.2
EUR	GBR-TSI	100	38.9
EUR	CEU-TSI	57	53.8
EUR	CEU-GBR	17	14.3
CON	AFR-EUR	348	52.2
CON	AFR-ASN	317	52.6
CON	ASN-EUR	190	53.4

Table S12A Summary of sites showing high levels of population differentiation

- Not a single variant 100% unique to a given population
- 17% of low-frequency variants (.5-5% pop. freq) observed in a single ancestry group
- 50% of rare variants (<.5%) observed in a single population

Variation across populations



A draft human pangenome reference

Liao et al (2023) Nature. <https://doi.org/10.1038/s41586-023-05896-x>

A Systematic Survey of Loss-of-Function Variants in Human Protein-Coding Genes

Daniel G. MacArthur,^{1,2*} Suganthi Balasubramanian,^{3,4} Adam Frankish,¹ Ni Huang,¹ James Morris,¹ Klaudia Walter,¹ Luke Jostins,¹ Lukas Habegger,^{3,4} Joseph K. Pickrell,⁵ Stephen B. Montgomery,^{6,7} Cornelis A. Albers,^{1,8} Zhengdong D. Zhang,⁹ Donald F. Conrad,¹⁰ Gerton Lunter,¹¹ Hancheng Zheng,¹² Qasim Ayub,¹ Mark A. DePristo,¹³ Eric Banks,¹³ Min Hu,¹ Robert E. Handsaker,^{13,14} Jeffrey A. Rosenfeld,¹⁵ Menachem Fromer,¹³ Mike Jin,³ Xinmeng Jasmine Mu,^{3,4} Ekta Khurana,^{3,4} Kai Ye,¹⁶ Mike Kay,¹ Gary Ian Saunders,¹ Marie-Marthe Suner,¹ Toby Hunt,¹ If H. A. Barnes,¹ Clara Amid,^{1,17} Denise R. Carvalho-Silva,¹ Alexandra H. Bignell,¹ Catherine Snow,¹ Bryndis Yngvadottir,¹ Suzannah Bumpstead,¹ David N. Cooper,¹⁸ Yali Xue,¹ Irene Gallego Romero,^{1,5} 1000 Genomes Project Consortium, Jun Wang,¹² Yingrui Li,¹² Richard A. Gibbs,¹⁹ Steven A. McCarroll,^{13,14} Emmanouil T. Dermitzakis,⁷ Jonathan K. Pritchard,^{5,20} Jeffrey C. Barrett,¹ Jennifer Harrow,¹ Matthew E. Hurles,¹ Mark B. Gerstein,^{3,4,21†} Chris Tyler-Smith^{1†}

Genome-sequencing studies indicate that all humans carry many genetic variants predicted to cause loss of function (LoF) of protein-coding genes, suggesting unexpected redundancy in the human genome. Here we apply stringent filters to 2951 putative LoF variants obtained from 185 human genomes to determine their true prevalence and properties. **We estimate that human genomes typically contain ~100 genuine LoF variants with ~20 genes completely inactivated.** We identify rare and likely deleterious LoF alleles, including 26 known and 21 predicted severe disease-causing variants, as well as common LoF variants in nonessential genes. We describe functional and evolutionary differences between LoF-tolerant and recessive disease genes and a method for using these differences to prioritize candidate genes found in clinical sequencing studies.

(2012) Science. doi: 10.1126/science.1215040

Homozygous LoF Mutations

LETTER

doi:10.1038/nature22034

Human knockouts and phenotypic analysis in a cohort with a high rate of consanguinity

Danish Saleheen^{1,2*}, Pradeep Natarajan^{3,4*}, Irina M. Armean^{5,6}, Wei Zhao¹, Asif Rasheed⁷, Sumeet A. Khetarpal⁸, Hong-Hee Won⁷, Konrad J. Karczewski^{1,5}, Anne H. O'Donnell¹, Luria^{1,5,8}, Kaitlin E. Samocha^{1,5}, Benjamin Weisburd^{1,5}, Namrata Gupta⁹, Moazzam Zaidi⁷, Maria Samuel⁷, Atif Imran⁷, Shahid Abbas⁸, Faisal Majeed⁷, Madiha Ishaq⁷, Saba Akhtar⁷, Kevin Trindade⁶, Megan Mucksavage⁶, Nadeem Qamar¹⁰, Khan Shah Zaman¹⁰, Zia Yaqoob¹⁰, Tahir Saghir¹⁰, Syed Nadeem Hasan Rizvi¹⁰, Anis Memon¹⁰, Nadeem Hayyat Mallick¹¹, Mohammad Ishaq¹², Syed Zahed Rasheed¹², Fazal-ur-Rehman Memon¹³, Khalid Mahmood¹⁴, Naveeduddin Ahmed¹⁵, Ron Do^{16,17}, Ronald M. Krauss¹⁸, Daniel G. MacArthur^{1,5}, Stacey Gabriel⁴, Eric S. Lander⁴, Mark J. Daly^{4,5}, Philippe Frossard^{2,8}, John Danesh^{19,20}, Daniel J. Rader^{6,21} & Sekar Kathiresan^{1,4}

A major goal of biomedicine is to understand the function of every gene in the human genome¹. Loss-of-function mutations can disrupt both copies of a given gene in humans and phenotypic analysis of such 'human knockouts' can provide insight into gene function. Consanguineous unions are more likely to result in offspring carrying homozygous loss-of-function mutations. In Pakistan, consanguinity rates are notably high². Here we sequence the protein-coding regions of 10,503 adult participants in the Pakistan Risk of Myocardial Infarction Study (PROMIS), designed to understand the determinants of cardiometabolic diseases in individuals from South Asia³. We identified individuals carrying homozygous predicted loss-of-function (pLoF) mutations, and performed phenotypic analysis involving more than 200 biochemical and disease traits. We enumerated 49,138 rare (<1% minor allele frequency) pLoF mutations. These pLoF mutations are estimated to knock out 1,317 genes, each in at least one participant. Homozygosity for pLoF mutations at *PLA2G7* was associated with absent enzymatic activity of soluble lipoprotein-associated phospholipase A2; at *CYP2F1*, with higher plasma interleukin-8 concentrations; at *TREH*, with lower concentrations of apoB-containing lipoprotein subfractions; at either *A3GALT2* or *NRG4*, with markedly reduced plasma insulin C-peptide concentrations; and at *SLC9A3R1*, with mediators of calcium and phosphate signalling. Heterozygous deficiency of *APOC3* has been shown to protect against coronary heart disease^{4,5}; we identified *APOC3* homozygous pLoF carriers in our cohort. We recruited these human knockouts and challenged them with an oral fat load. Compared with family members lacking the mutation, individuals with *APOC3* knocked out displayed marked blunting of the usual post-prandial rise in plasma triglycerides. Overall, these observations provide a roadmap for a 'human knockout project', a systematic effort to understand the phenotypic consequences of complete disruption of genes in humans.

Across all participants (Table 1), exome sequencing yielded 1,639,223 exonic and splice-site sequence variants in 19,026 autosomal genes that passed initial quality control metrics. Of these, 57,137 mutations

across 14,345 autosomal genes were annotated as pLoF mutations (that is, nonsense, frameshift, or canonical splice-site mutations predicted to inactivate a gene). To increase the probability that mutations are correctly annotated as pLoF by automated algorithms, we removed nonsense and frameshift mutations occurring within the last 5% of the transcript and within exons flanked by non-canonical splice sites, splice-site mutations at small (<15 bp) introns, at non-canonical splice sites, and where the purported pLoF allele is observed across primates. Common pLoF alleles are less likely to exert strong functional effects as they are less constrained by purifying selection; thus, we define pLoF mutations in the rest of the manuscript as variants with a minor allele frequency (MAF) of <1% and passing the aforementioned bioinformatic filters. Applying these criteria, we generated a set of 49,138 pLoF mutations across 13,074 autosomal genes. The site-frequency spectrum for these pLoF mutations revealed that the majority was seen only in one or a few individuals (Extended Data Fig. 1).

Across all 10,503 PROMIS participants, both copies of 1,317 distinct genes were predicted to be inactivated owing to pLoF mutations. A full listing of all 1,317 genes knocked out, the number of knockout participants for each gene, and the specific pLoF mutation(s) are provided in Supplementary Table 1. 891 (67.7%) of the genes were knocked out only in one participant (Fig. 1a). Nearly 1 in 5 of the participants that were sequenced (1,843 individuals, 17.5%) had at least one gene knocked out by a homozygous pLoF mutation. 1,504 of these 1,843 individuals (81.6%) were homozygous pLoF carriers for just one gene, but the minority of participants had more than one gene knocked out and one participant had six genes with homozygous pLoF genotypes.

We compared the coefficient of inbreeding (*F* coefficient) in PROMIS participants with that of 15,249 individuals from outbred populations of European or African American ancestry. The *F* coefficient estimates the excess homozygosity compared with an outbred ancestor. PROMIS participants had a fourfold higher median inbreeding coefficient compared to outbred populations (0.016 versus 0.0041; $P < 2 \times 10^{-10}$) (Fig. 1b). Additionally, those in PROMIS who reported that their parents were closely related had even higher median inbreeding coefficients than

- Homozygous LoF mutations are rare in most people, but enriched in people born from consanguineous relationships
- Sequence the exomes of many such people, find their homozygous LoFs, relate to 200 biochemical or disease traits
- A “natural” experiment to understand what genes do: people with both copies of *APOC3* disabled can clear fat from their bloodstream much faster than others, suggests we should develop compounds to prevent heart attacks

(2017) Nature. doi:10.1038/nature22034